



Universidad Nacional Experimental de Guayana.

Vicerrectorado Académico.

Coordinación General de Pregrado.

Proyecto de Carrera: Ingeniería en Informática.

Unidad Curricular: Lenguajes y Compiladores.

Semestre 7 – Sección: 1 y 2.

Los Lenguajes de Programación.

Los ASTronautas

“Explorando el Universo de los Lenguajes, un Nodo a la vez”

Profesor:

Ing. Félix Márquez

Estudiantes:

C.I.: V-25.933.680. Albuquerque Sheen

C.I.: V-30.907.427. Antoima Mariangel

C.I.: V-28.475.271. García Carlos

C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, Junio del 2026.

Índice.

Resumen Académico	3
Introducción	4
2.1 Paradigmas.....	5
2.2 Propósito.....	8
2.3. Estructuras Morfológicas.....	12
2.4.Estructuras Sintácticas	16
Actividad I	24
Actividad II	33
Actividad III	64
Conclusión.	72
Referencias.....	74

Resumen Académico

Este estudio realiza un análisis comparativo de los lenguajes Zig, Python, Rust y JavaScript, evaluando sus paradigmas de programación, estructuras morfológicas y sintácticas, así como su rendimiento en procesamiento intensivo. Se empleó un benchmark basado en la Conjetura de Collatz ($3n+1$) para un rango de $N=100,000$ números, acumulando el total de pasos (10.753.840 verificado en los cuatro lenguajes), midiendo tiempos de ejecución y consumo de memoria pico. Los resultados muestran a JavaScript (Node.js) como el más rápido con 22 ms, atribuible a su motor V8 con compilación Just-In-Time (JIT), seguido por Rust con 26 ms, gracias a su sistema de ownership sin recolector de basura. Zig alcanzó 47 ms priorizando el control explícito sobre optimizaciones automáticas, mientras que Python resultó significativamente más lento con 895 ms (aproximadamente 40 veces más que JavaScript), debido a su naturaleza interpretada (CPython), tipado dinámico y la presencia del GIL (Global Interpreter Lock).

El análisis morfológico reveló que la indentación significativa de Python complejiza el análisis léxico al hacer significativos los espacios al inicio de línea, mientras que las llaves obligatorias en Rust y Zig simplifican el proceso de tokenización. Como innovación principal, se diseñó el Lenguaje L, un Lenguaje de Dominio Específico (DSL) para sistemas de microrredes eléctricas críticas denominado ECO-GRID. Su gramática independiente de contexto no ambigua y su léxico especializado permiten abstraer la complejidad del hardware subyacente, ofreciendo primitivas seguras para control termal (prevención de fugas térmicas), balance autónomo de carga eléctrica y gestión de alarmas con acumulador de criticidad. Se concluye que no existe un lenguaje universalmente superior; la selección estratégica basada en el dominio de aplicación es crucial. El DSL propuesto demuestra cómo la teoría de autómatas y lenguajes formales se aplica efectivamente para crear interfaces hombre-máquina predecibles y seguras en entornos de alto riesgo operativo.

Introducción

El diseño e implementación de software se fundamenta en una relación triádica entre el ser humano, el lenguaje de programación y la máquina subyacente. En el centro de esta relación se sitúa un problema estructural de larga data: ¿cómo traducir el pensamiento lógico y algorítmico del programador en instrucciones eficientes que la máquina pueda ejecutar sin ambigüedad ni pérdida de significado? La respuesta no reside únicamente en la elección de un lenguaje u otro, sino en la comprensión profunda de los niveles jerárquicos que constituyen todo lenguaje formal. El nivel morfológico define cómo los caracteres se agrupan en tokens mediante expresiones regulares y autómatas finitos. El nivel sintáctico establece, a través de gramáticas independientes del contexto (CFG) en notación BNF, cómo esos tokens se combinan para formar sentencias y programas completos, garantizando la no ambigüedad estructural. Finalmente, el nivel semántico asigna significado a las construcciones válidas, completando el proceso de análisis que todo compilador o intérprete debe ejecutar.

La relevancia actual de analizar la eficiencia estructural del software se ha intensificado por dos fenómenos convergentes. Por un lado, el estancamiento de la Ley de Moore y el auge de los procesadores multicore han desplazado el óptimo de rendimiento desde la velocidad bruta del hardware hacia la eficiencia de las abstracciones lingüísticas y la concurrencia segura. Por otro lado, la proliferación de lenguajes multiparadigma (Python, Rust, JavaScript y Zig) ha disuelto los límites tradicionales entre estilos imperativo, funcional, orientado a objetos y concurrente, exigiendo al ingeniero comprender cómo las decisiones de diseño morfológico y sintáctico impactan la compilación y la ejecución final.

El propósito del presente estudio es triple. Primero, se realiza un análisis teórico de cuatro lenguajes contemporáneos (Zig, Python, Rust y JavaScript) a través de sus paradigmas fundamentales, su estructura morfológica (palabras reservadas, identificadores, literales) y su estructura sintáctica (funciones, condicionales y bucles), documentada mediante BNF. Segundo, como validación empírica, se aborda la Conjetura de Collatz para 100,000 números, estableciendo un benchmark de tiempos de ejecución y consumo de memoria. Tercero, como culminación integradora, se presenta el diseño conceptual del Lenguaje L, un DSL para sistemas de microrredes eléctricas críticas (ECO-GRID), donde se aplican las técnicas de análisis léxico y sintáctico para construir una interfaz hombre-máquina segura, expresiva y libre de ambigüedades en un entorno de alto riesgo operativo.

2.1 Paradigmas

El concepto de paradigma de programación fue popularizado por Robert W. Floyd en su discurso de aceptación del Premio Turing en 1978, y posteriormente estructurado por Robert Harper en sus trabajos sobre teoría de lenguajes. Un paradigma de programación se define como un estilo fundamental de construcción de software que impone una determinada forma de pensar y organizar la solución computacional de un problema. Según Abelson y Sussman (1996), "un paradigma es un conjunto de conceptos, técnicas y una cosmovisión compartida por una comunidad de programadores que influye en cómo se conciben y resuelven los problemas" (p. 45). Esta definición implica que el paradigma no es meramente una cuestión sintáctica, sino una *filosofía de construcción*.

Clasificación de los Paradigmas Fundamentales

La literatura especializada reconoce cinco paradigmas principales, cada uno con sus principios axiomáticos y modelos de computación subyacentes.

2.1.1. Paradigma Imperativo

El paradigma imperativo es el más antiguo y el más cercano a la arquitectura de von Neumann. Se fundamenta en la máquina de estados con memoria modificable. Según Ortega de la Puente, Alfonseca, de la Cruz y Pulido (2006), "la programación imperativa se basa en la ejecución secuencial de instrucciones que modifican el estado del programa a través de asignaciones" (p. 78).

Principios característicos:

- El programa es una secuencia ordenada de sentencias.
- Las variables son celdas de memoria cuyo valor puede cambiar.
- El flujo de control se modifica mediante estructuras como condicionales (if-then-else) y bucles (while, for).
- La asignación ($=$, $:=$) es la operación fundamental.

Su principal virtud es la eficiencia y el control directo sobre el hardware, siendo la base de lenguajes como C, Fortran y Pascal. Su principal limitación es la propensión a efectos laterales y la dificultad para razonar formalmente sobre programas complejos.

2.1.2. Paradigma Orientado a Objetos

El paradigma orientado a objetos (POO) surge como una evolución del imperativo, incorporando conceptos del modelado del mundo real. Según Booch (1994), "la programación orientada a objetos es un método de implementación en el que los programas se organizan como

colecciones cooperativas de objetos, cada uno de los cuales representa una instancia de una clase, siendo las clases unidades de herencia y polimorfismo" (p. 33).

Principios nucleares:

- **Encapsulamiento:** Los datos y los métodos que operan sobre ellos se agrupan en una entidad llamada objeto, ocultando los detalles internos.
- **Herencia:** Mecanismo que permite definir una nueva clase a partir de otra existente, reutilizando y extendiendo su comportamiento.
- **Polimorfismo:** Capacidad de que diferentes objetos respondan de forma específica al mismo mensaje o invocación.

Lenguajes representativos incluyen Smalltalk (el pionero), Java, C++, C# y Python (en su vertiente orientada a objetos). La POO destaca en la gestión de grandes sistemas modulares y en la reutilización de código, aunque introduce una complejidad conceptual significativa.

2.1.3. Paradigma Funcional

El paradigma funcional encuentra sus raíces en el cálculo lambda, desarrollado por Alonzo Church en la década de 1930. Este paradigma trata la computación como la evaluación de funciones matemáticas puras, evitando por completo el estado mutable y los efectos laterales. Según Bird (2014), "la programación funcional es un estilo de programación en el que la única forma de computar es mediante la aplicación de funciones a argumentos, y donde las funciones son ciudadanos de primera clase, pudiendo ser pasadas como argumentos, devueltas como resultados y almacenadas en estructuras de datos" (p. 12).

Propiedades fundamentales:

- **Transparencia referencial:** Una expresión puede ser reemplazada por su valor sin alterar el comportamiento del programa. Esto facilita el razonamiento formal y la verificación.
- **Inmutabilidad:** Las variables, una vez asignadas, no pueden modificarse. Las "actualizaciones" generan nuevas estructuras.
- **Funciones de orden superior:** Funciones que toman otras funciones como parámetros o las devuelven como resultado (ej. map, filter, fold).
- **Evaluación perezosa** (en lenguajes como Haskell): Las expresiones no se evalúan hasta que sus valores son realmente necesarios.

Los lenguajes funcionales puros incluyen Haskell y Miranda. Lenguajes multiparadigma que incorporan características funcionales son Python, JavaScript, Scala y F#. El paradigma funcional es especialmente adecuado para sistemas concurrentes y para razonamiento formal, aunque puede resultar menos intuitivo para programadores acostumbrados al estilo imperativo.

2.1.4. Paradigma Lógico

El paradigma lógico, cuyo exponente máximo es Prolog (Programmation en Logique), se basa en la lógica de primer orden. Según Clocksin y Mellish (2003), "un programa lógico consiste en un conjunto de cláusulas de Horn, que son implicaciones lógicas de la forma $A \leftarrow B_1, B_2, \dots, B_n$, donde A es la cabeza y los B_i son los objetivos. La ejecución consiste en un mecanismo de resolución SLD (Selective Linear Definite clause resolution)" (p. 7).

Mecanismo de funcionamiento:

- El programador declara hechos (ej. padre(juan, maria).) y reglas (ej. abuelo(X,Y) :- padre(X,Z), padre(Z,Y).).
- El sistema posee un motor de inferencia basado en unificación y backtracking.
- La consulta (ej. ?- abuelo(luis, maria).) se resuelve intentando demostrar la existencia de una instanciación de variables que satisfaga la expresión.

Este paradigma es declarativo por excelencia: se especifica *qué* se quiere demostrar, no *cómo* hacerlo. Aunque su eficiencia es menor que la de los paradigmas imperativos para problemas de propósito general, resulta insuperable en ámbitos como sistemas expertos, procesamiento de lenguaje natural y resolución de restricciones.

2.1.5. Paradigma Concurrente y Basado en Actores

La concurrencia es una propiedad intrínseca de los sistemas modernos (múltiples núcleos, arquitecturas distribuidas). El modelo de actores, propuesto por Carl Hewitt en 1973, es uno de los enfoques más influyentes para gestionar la concurrencia. Según Agha (1986), "un actor es una entidad computacional que posee un comportamiento y un estado propio; la comunicación entre actores se realiza exclusivamente mediante el envío de mensajes asíncronos. Cada actor procesa los mensajes de su buzón de entrada uno a la vez, eliminando la necesidad de mecanismos explícitos de sincronización como semáforos o monitores" (p. 15).

Principios del modelo de actores:

- Los actores son unidades autónomas y ligeras.
- El estado de un actor es privado y solo puede ser modificado por sí mismo.
- Al recibir un mensaje, un actor puede:
 - ❖ Modificar su estado interno.
 - ❖ Enviar mensajes a otros actores (cuyas direcciones conoce).
 - ❖ Crear nuevos actores.
- No existe memoria compartida, lo que elimina condiciones de carrera.

Erlang (diseñado por Ericsson para sistemas de telecomunicaciones) y Elixir son los lenguajes más representativos del modelo de actores. En el ecosistema JVM, el framework Akka implementa este modelo para Scala y Java.

Lenguajes Multiparadigma y la Necesidad del Dominio Plural

Los lenguajes de programación contemporáneos han abandonado la pureza paradigmática en favor del multiparadigma. Python, por ejemplo, soporta simultáneamente estilos imperativo, orientado a objetos y funcional. Scala integra la orientación a objetos (con clases y herencia) con el paradigma funcional (inmutabilidad, funciones de orden superior y pattern matching). Kotlin, Swift y Rust siguen tendencias similares. Como señala Sebesta (2016), "ningún paradigma es superior en todos los dominios de aplicación. La habilidad del ingeniero moderno consiste en conocer las fortalezas y debilidades de cada paradigma y seleccionar el adecuado según las restricciones del problema, los requisitos de rendimiento, la mantenibilidad esperada y las competencias del equipo" (p. 156).

De esta reflexión deriva la máxima ya enunciada: aunque es natural desarrollar una preferencia personal por un paradigma (la comodidad cognitiva y la experiencia acumulada inclinan al profesional hacia un estilo), la excelencia técnica exige dominar los demás y aplicarlos con criterio contextual. Un algoritmo de transformación de datos puede ser más claro en estilo funcional; un sistema de gestión de entidades con relaciones complejas, más expresivo en orientación a objetos; un proceso de control de hardware, más eficiente en imperativo; y un sistema de reglas de negocio dinámicas, más adecuado en lógico.

2.2 Propósito

El propósito de un lenguaje de programación se refiere a la intencionalidad de diseño que guía su creación y que determina el conjunto de problemas para los cuales el lenguaje resulta más adecuado. Según Pratt y Zelkowitz (2001), "el propósito de un lenguaje es la respuesta a la pregunta fundamental: ¿para qué fue diseñado este lenguaje? Esta cuestión condiciona todas las decisiones posteriores sobre sintaxis, semántica, sistema de tipos y paradigmas soportados" (p. 23).

Es importante distinguir entre:

- **Propósito generalista:** Lenguajes concebidos para abordar una amplia gama de problemas sin especialización dominante.
- **Propósito específico** (Domain-Specific Languages, DSL): Lenguajes optimizados para un dominio de aplicación concreto.

2.2.1. El Propósito como Fuerza de Diseño

Cualquier lenguaje de programación responde a un contexto histórico y a una necesidad concreta. Analizar el propósito permite comprender por qué ciertos lenguajes presentan características aparentemente limitantes o inusuales.

Ejemplos paradigmáticos de propósito fundacional:

- Fortran (Formula Translation, 1957) fue diseñado explícitamente para computación científica y matemática. Por ello, incorporó desde su origen tipos de datos para matrices y números complejos, así como una sintaxis cercana a la notación algebraica. Su propósito específico explica su eficiencia histórica en cálculos numéricos intensivos.
- C (Dennis Ritchie, 1972) fue concebido para la programación de sistemas operativos, específicamente para reescribir UNIX. Su propósito explicaba la necesidad de operadores de bajo nivel (punteros, aritmética de direcciones, manipulación directa de bits), una mínima abstracción del hardware y una estrecha correspondencia con el ensamblador.
- Lisp (John McCarthy, 1958) nació como una notación formal para el cálculo lambda y la manipulación simbólica. Su propósito en inteligencia artificial y procesamiento de listas explica sus características distintivas: homogeneidad sintáctica (código como datos), recolector de basura temprano y funciones como ciudadanos de primera clase.
- SQL (1974) fue diseñado específicamente para la gestión y consulta de bases de datos relacionales. Su propósito declarativo explica por qué el programador especifica *qué* datos recuperar, no *cómo* hacerlo, dejando la optimización al motor de la base de datos.

Según Loudon y Lambert (2012), "cuando un lenguaje se utiliza fuera de su propósito original, se experimentan tensiones de diseño: el lenguaje exhibe verbosidad, requiere patrones de codificación repetitivos (idioms), o necesita extensiones no previstas que complican el compilador y el aprendizaje" (p. 98). Esto explica fenómenos como el uso de Java para scripts rápidos (tarea para la que Python o Ruby son más adecuados) o el uso de Python para sistemas embebidos con recursos críticos (dominio tradicional de C).

2.2.2. Propósito y Abstracción

El propósito de un lenguaje está íntimamente ligado a su nivel de abstracción y al tipo de abstracciones que ofrece:

- **Lenguajes de bajo nivel** (ensamblador): Su propósito es proporcionar control preciso sobre el hardware, sin abstracciones que oculten el funcionamiento real de la máquina. Son adecuados para controladores de dispositivos, kernels de sistemas operativos y código crítico en tiempo real.
- **Lenguajes de medio nivel** (C, C++): Proporcionan abstracciones sobre la máquina subyacente (tipos, estructuras, funciones) pero permiten escapar de ellas cuando es necesario. Su propósito equilibra portabilidad y control.

- **Lenguajes de alto nivel** (Python, Java, C#): Su propósito es maximizar la productividad del programador, la legibilidad y la portabilidad, sacrificando cierto grado de control sobre el hardware. Automatizan la gestión de memoria (recolección de basura) y proporcionan ricas bibliotecas estándar.
- **Lenguajes de muy alto nivel** (Haskell, Prolog, MATLAB, R): Su propósito es la expresividad máxima en dominios específicos (razonamiento formal, lógica simbólica, cómputo matricial, estadística). A menudo son interpretados o cuentan con optimizaciones muy sofisticadas.

Como señala Sebesta (2016), "la elección del lenguaje más adecuado para un proyecto debe comenzar con un análisis de propósito: ¿cuáles son los dominios de problema que se resolverán? ¿Qué restricciones de rendimiento existen? ¿Qué plazos de desarrollo? ¿Qué equipo humano estará disponible?" (p. 312).

2.2.3. El Propósito en el Contexto del Benchmarking Técnico

Dentro del marco de este informe, el propósito adquiere una dimensión adicional: la necesidad de desarrollar competencias de benchmarking técnico y análisis comparativo. Comprender el propósito de cada lenguaje permite interpretar correctamente las mediciones de rendimiento.

Consideraciones para la medición de tiempos de ejecución:

1. **Propósito y eficiencia esperada:** No tiene sentido comparar el rendimiento bruto de Python (diseñado para productividad y legibilidad) con el de C (diseñado para control y eficiencia). El propósito de Python prioriza el tiempo de desarrollo sobre el tiempo de ejecución.
2. **Optimizaciones específicas del dominio:** MATLAB es más lento que C en bucles explícitos, pero extremadamente rápido en operaciones matriciales porque su propósito (cómputo numérico) ha guiado a sus implementadores a optimizar intensamente las operaciones vectorizadas.
3. **Compilación vs. interpretación como consecuencia del propósito:** Lenguajes compilados (C, Rust, Go) buscan eficiencia en tiempo de ejecución; lenguajes interpretados (Python, Ruby, JavaScript original) priorizan la interactividad y la facilidad de desarrollo. Las implementaciones híbridas (JIT, como Java HotSpot o V8) intentan conciliar ambos extremos.

Sebesta (2016) advierte además: "Una comparación ingenua de tiempos de ejecución puede llevar a conclusiones erróneas si no se considera que cada lenguaje implementa ciertas optimizaciones que favorecen los problemas para los que fue diseñado. Un benchmark diseñado para favorecer el estilo imperativo penalizará injustamente a un lenguaje funcional puro, y viceversa" (p. 456).

2.2.4. El Propósito como Criterio de Selección en Entornos Multiparadigma

En lenguajes multiparadigma modernos (Python, Scala, Kotlin, Swift), el propósito no es monolítico para todo el lenguaje, sino que puede ser modular según el subsistema que se esté implementando. Esto eleva la responsabilidad del programador: no solo debe seleccionar el lenguaje adecuado, sino también, dentro de un mismo lenguaje, elegir el estilo paradigmático apropiado para cada módulo.

Ejemplo concreto:

- Una aplicación de procesamiento de datos en Python podría usar estilo funcional (map, filter, reduce) para las transformaciones del pipeline central (por claridad y facilidad de paralelización).
- El mismo programa podría usar estilo imperativo con bucles para las rutas críticas en tiempo real (por predicibilidad del rendimiento).
- La interfaz gráfica podría implementarse bajo el paradigma orientado a objetos (herencia de widgets, encapsulamiento de eventos).

El propósito, en este contexto, se vuelve una decisión de *arquitectura interna*, no solo de selección inicial del lenguaje.

2.2.5. Propósito y Evolución Histórica

El propósito de un lenguaje puede mutar con el tiempo. Originalmente, JavaScript fue concebido como un lenguaje de scripting para validación de formularios en navegadores (propósito modesto y de ámbito restringido). Su propósito actual, con Node.js, incluye servidores web, aplicaciones de tiempo real, herramientas de línea de comandos y sistemas embebidos. Esta expansión de propósito ha exigido continuas revisiones del estándar ECMAScript y mejoras masivas en los motores de ejecución (V8, SpiderMonkey, JavaScriptCore).

De manera similar, Python, inicialmente pensado como un lenguaje de scripting para tareas de administración de sistemas y enseñanza, ha visto expandido su propósito hacia el análisis de datos (con Pandas, NumPy), machine learning (TensorFlow, PyTorch), computación científica (SciPy) y desarrollo web (Django, Flask).

Como indica Harper (2016), "la evolución del propósito de un lenguaje es un testimonio de su flexibilidad conceptual, pero también una fuente de fricción, pues las características añadidas posteriormente pueden no integrarse limpiamente con el diseño original. Los lenguajes que manejan bien esta evolución son aquellos con una semántica fundamental sólida y una comunidad comprometida con la calidad de las extensiones" (p. 87).

2.3. Estructuras Morfológicas

La estructura morfológica de un lenguaje de programación, también denominada *estructura léxica*, se refiere al conjunto de reglas que determinan cómo los caracteres individuales se agrupan en unidades atómicas con significado, denominadas lexemas o tokens. Según Ortega de la Puente, Alfonseca, de la Cruz y Pulido (2006), "la morfología es el nivel más bajo del análisis de un lenguaje; se ocupa de la composición de las palabras y símbolos que constituyen el vocabulario del lenguaje, ignorando temporalmente su organización jerárquica o su significado contextual" (p. 67).

El análisis morfológico constituye la primera fase de todo compilador o intérprete, y su correcta definición es condición necesaria (aunque no suficiente) para que un lenguaje sea procesable automáticamente.

2.3.1. Conceptos Fundamentales del Análisis Morfológico

El Lexema y el Token

Es necesario distinguir dos conceptos íntimamente relacionados pero no idénticos:

- **Lexema:** Secuencia concreta de caracteres que aparece en el código fuente. Por ejemplo, la secuencia "contador123" es un lexema.
- **Token:** Categoría abstracta que clasifica un conjunto de lexemas. Por ejemplo, el token IDENTIFICADOR clasifica todos los lexemas que cumplen las reglas de formación de nombres de variables.

Según Aho, Lam, Sethi y Ullman (2007), "el analizador léxico no devuelve lexemas, sino pares (token, atributo), donde el token indica la categoría sintáctica y el atributo lleva información adicional sobre el lexema concreto que se ha reconocido, como su valor numérico o su entrada en la tabla de símbolos" (p. 112).

Componentes de la Estructura Morfológica

La morfología de un lenguaje típico incluye las siguientes categorías de tokens:

- **Palabras clave (keywords):** Son lexemas con significado fijo e inalterable que el lenguaje reserva para usos específicos. Ejemplos: if, while, class, def, return. En la mayoría de los lenguajes modernos, las palabras clave no pueden ser usadas como identificadores de usuario. Lenguajes antiguos como Fortran no tenían esta restricción, lo que generaba ambigüedades en el análisis.
- **Identificadores:** Son nombres que el programador asigna a variables, funciones, clases, tipos u otras entidades. Suelen seguir reglas específicas: comienzan con una letra o guion bajo, seguida de letras, dígitos o guiones bajos. Louden y Lambert (2012) señalan que "la definición de la morfología de los identificadores influye directamente en la legibilidad:

permitir guiones (mi-variable) mejora la legibilidad pero complica el análisis porque el guion también puede ser un operador de resta" (p. 156).

- **Constantes y literales:** Representan valores fijos en el código. Pueden ser numéricos (enteros 42, reales 3.1416), booleanos (true, false), de carácter ('a') o de cadena ("Hola mundo"). Cada tipo de literal tiene su propia morfología: las cadenas, por ejemplo, requieren reglas para manejar caracteres de escape (\n, \t, \").
- **Operadores:** Símbolos que representan operaciones. Pueden ser unarios (-, +, !), binarios (+, -, *, /, ==, <=, &&) o ternarios (? : en C y derivados). Algunos operadores son palabras clave, como and, or, not en Python.
- **Símbolos de puntuación y delimitadores:** Incluyen paréntesis (), llaves { }, corchetes [], punto y coma ;, comas , y otros caracteres que sirven para estructurar el código sin representar una operación.
- **Comentarios:** Aunque técnicamente no generan tokens, su morfología debe ser definida para que el analizador léxico los identifique y los ignore. Pueden ser de línea (// en C++/Java/JavaScript) o de bloque (/* ... */).

2.3.2. Expresiones Regulares como Herramienta Formal

La herramienta matemática estándar para definir la morfología de un lenguaje son las expresiones regulares. Según Hopcroft, Motwani y Ullman (2007), "una expresión regular es una notación algebraica para describir conjuntos de cadenas sobre un alfabeto dado, utilizando operadores de concatenación, unión y clausura de Kleene" (p. 94).

Operadores básicos de las expresiones regulares:

Operación	Notación común	Significado
Concatenación	AB	Una A seguida de una B
Unión (alternativa)	A B	Una A o una B
Clausura de Kleene	A*	Cero o más repeticiones de A
Clausura positiva	A+	Una o más repeticiones de A
Opcional	A?	Cero o una A

Definición morfológica de tokens mediante expresiones regulares (ejemplos):

- IDENTIFICADOR $\rightarrow [A-Za-z_][A-Za-z0-9_]*$
(Una letra o guion bajo, seguida de cero o más letras, dígitos o guiones bajos)
- NUMERO_ENTERO $\rightarrow [0-9]^+$
(Uno o más dígitos)

- NUMERO_REAL $\rightarrow [0-9]+\.[0-9]^+$
(Uno o más dígitos, un punto decimal, uno o más dígitos)
- CADENA $\rightarrow "[^"]^+"$
(Comillas dobles, seguida de cero o más caracteres que no sean comillas dobles, y comillas dobles finales)
- COMENTARIO_LINEA $\rightarrow //[^\n]^*$
(Dos barras, seguidas de cero o más caracteres que no sean salto de línea)
- ESPACIO_BLANCO $\rightarrow [\t\n\r]^+$
(Espacios, tabuladores, saltos de línea)

Como señala Sebesta (2016), "las expresiones regulares son el fundamento de herramientas generadoras de analizadores léxicos como Lex (Unix) o su versión GNU Flex. Estas herramientas toman una especificación de patrones asociados a acciones y producen automáticamente el código de un analizador léxico eficiente" (p. 134).

2.3.3. El Proceso de Análisis Morfológico

El análisis morfológico se realiza mediante un autómata finito determinista (DFA) o no determinista (NFA), que recorre el código fuente carácter por carácter. El proceso sigue estos pasos:

1. Escanear el flujo de entrada, leyendo caracteres secuencialmente.
2. Acumular caracteres en un buffer mientras se reconoce el patrón de algún token.
3. Identificar el token más largo posible que coincida con algún patrón definido (regla del *maximal munch*).
4. Retornar un par (token, atributo) al analizador sintáctico.
5. Ignorar los caracteres que no generan tokens (espacios, tabuladores, comentarios).
6. Reportar errores si se encuentra un carácter que no da inicio a ningún token válido.

Según Ortega de la Puente et al. (2006), "el algoritmo de análisis léxico debe resolver ambigüedades cuando dos patrones coinciden con el mismo prefijo. La solución habitual es priorizar los patrones más largos y, en caso de empate, el que aparezca primero en la especificación" (p. 89).

Ejemplo de ambigüedad resuelta por prioridad:

- Si `if` es palabra clave y también podría ser IDENTIFICADOR (porque comienza con letra), el analizador debe dar prioridad a `if` como palabra clave. En la práctica, esto se implementa procesando primero las palabras clave y solo después los identificadores genéricos.

2.3.4. La Tabla de Símbolos

Una estructura de datos fundamental asociada al análisis morfológico es la tabla de símbolos. Según Aho et al. (2007), "la tabla de símbolos es una estructura de datos que contiene un registro por cada identificador (variable, función, clase, etc.) que aparece en el programa. Cada registro almacena información como el lexema del identificador, su tipo, su ámbito (alcance) y, eventualmente, su dirección de memoria o posición relativa" (p. 126).

El analizador léxico generalmente inserta cada identificador que encuentra en la tabla de símbolos (si no está ya presente) y devuelve al analizador sintáctico un token IDENTIFICADOR acompañado de un puntero o índice a la entrada correspondiente en dicha tabla.

Características de una tabla de símbolos eficiente:

- Implementación mediante tablas hash para acceso en tiempo constante ($O(1)$).
- Soporte para ámbitos anidados (estructuras de pila que permiten ocultar identificadores en bloques internos).
- Almacenamiento del lexema solo una vez para ahorrar memoria (internamiento de cadenas).

2.3.5. Impacto de la Morfología en Compiladores e Interpretes

La complejidad y claridad de las reglas morfológicas afectan directamente el análisis posterior:

- **Eficiencia del análisis:** Una morfología simple (como la de C, con pocos tipos de tokens y reglas claras) permite implementar analizadores léxicos extremadamente rápidos, a menudo generados automáticamente con herramientas como Lex. Una morfología compleja (como la de C++, que debe distinguir entre `>>` como operador de desplazamiento y `>>` como cierre de dos plantillas anidadas) requiere técnicas más sofisticadas y puede ralentizar la compilación.
- **Claridad para el programador:** Una morfología intuitiva reduce errores de principiantes. Por ejemplo, Python permite guiones bajos en identificadores pero no guiones medios, una decisión que evita la ambigüedad con la resta. Ruby permite `$variable_global` y `@@variable_clase`, usando la morfología (el prefijo `$` o `@@`) para comunicar información semántica (ámbito de la variable) sin necesidad de declaraciones explícitas.
- **Portabilidad:** Los lenguajes que usan exclusivamente el alfabeto ASCII (o Unicode estándar) para su morfología son más portables que aquellos que dependen de caracteres específicos de ciertos sistemas operativos o codificaciones.
- **Extensibilidad:** Lenguajes como Lisp y Scheme tienen una morfología minimalista (pocos tipos de tokens, todo basado en paréntesis y átomos). Esta simplicidad facilita la

extensión del lenguaje y la implementación de macros que transforman código fuente, pero puede resultar críptica para programadores no habituados.

2.3.6. Herramientas para Generación de Analizadores Morfológicos

En el desarrollo práctico de compiladores e intérpretes, no se escriben analizadores léxicos manualmente (salvo en contextos educativos). Se utilizan generadores de analizadores léxicos:

- **Lex (original UNIX) y Flex (GNU versión):** Toman una especificación de patrones y acciones y generan código C para un analizador léxico.
- **JFlex:** Versión para Java.
- **PLY (Python Lex-Yacc):** Biblioteca Python que emula Lex y Yacc.
- **ANTLR:** Generador de analizadores (léxico y sintáctico) que soporta múltiples lenguajes objetivo (Java, C#, Python, JavaScript, etc.).

Estas herramientas permiten al diseñador de lenguajes concentrarse en la especificación de alto nivel (expresiones regulares) y delegar la implementación del autómata subyacente.

2.4. Estructuras Sintácticas

La estructura sintáctica de un lenguaje de programación se refiere al conjunto de reglas que determinan cómo los tokens (las unidades atómicas producidas por el análisis morfológico) pueden combinarse para formar sentencias, expresiones, bloques, funciones y programas completos. Según Aho, Lam, Sethi y Ullman (2007), "la sintaxis de un lenguaje especifica la forma correcta de las construcciones del lenguaje, independientemente de su significado. Un programa es sintácticamente correcto si existe una derivación completa desde el símbolo inicial de la gramática hasta la cadena de tokens que representa el programa" (p. 168). Mientras la morfología se ocupa de las *palabras* del lenguaje, la sintaxis se ocupa de las *oraciones* y de su estructura jerárquica. El análisis sintáctico (o *parsing*) es la segunda fase de todo compilador o intérprete y constituye el núcleo del proceso de comprensión del código fuente.

2.4.1. Fundamentos Formales: Gramáticas Independientes del Contexto

La herramienta matemática fundamental para describir la sintaxis de los lenguajes de programación son las gramáticas independientes del contexto (Context-Free Grammars, CFG), también denominadas gramáticas de tipo 2 en la jerarquía de Chomsky.

Según Hopcroft, Motwani y Ullman (2007), "una gramática independiente del contexto es una cuádrupla $G = (V, T, P, S)$, donde:

- V es un conjunto finito de símbolos no terminales (categorías sintácticas),
- T es un conjunto finito de símbolos terminales (los tokens),

- P es un conjunto finito de reglas de producción de la forma $A \rightarrow \alpha$, donde $A \in V$ y $\alpha \in (V \cup T)^*$,
- $S \in V$ es el símbolo inicial" (p. 185).

Ejemplo ilustrativo de una gramática simple:

Para una mini calculadora que admite sumas y multiplicaciones de números enteros:

- Terminales (T): NUMERO, +, *, (,)
- No terminales (V): E (expresión), T (término), F (factor)
- Reglas de producción (P):
 1. $E \rightarrow E + T$ (Una expresión puede ser una expresión más un término)
 2. $E \rightarrow T$ (Una expresión puede ser un término)
 3. $T \rightarrow T * F$ (Un término puede ser un término por un factor)
 4. $T \rightarrow F$ (Un término puede ser un factor)
 5. $F \rightarrow \text{NUMERO}$ (Un factor puede ser un número)
 6. $F \rightarrow (E)$ (Un factor puede ser una expresión entre paréntesis)
- Símbolo inicial (S): E

Esta gramática describe correctamente expresiones como $2 + 3 * 4$ o $(1 + 2) * 5$.

2.4.2. Notación BNF y sus Derivaciones

La notación más utilizada para expresar gramáticas de lenguajes de programación es la forma normal de Backus-Naur (BNF) y sus extensiones (EBNF, ABNF). Según Loudon y Lambert (2012), "BNF fue desarrollada por John Backus para describir la sintaxis de Algol 60 y posteriormente refinada por Peter Naur. Su principal contribución fue proporcionar una notación precisa, concisa y formal para especificar la sintaxis de los lenguajes, reemplazando descripciones informales en prosa que eran ambiguas e incompletas" (p. 204).

Correspondencia entre BNF y notación matemática:

Matemática	BNF	Significado	
$A \rightarrow \alpha \beta$	$\langle A \rangle ::= \alpha \beta$	A se expande como α seguido de β	
$A \rightarrow \alpha$	β	$\langle A \rangle ::= \alpha \mid \beta$	A se expande como α o como β
$A \rightarrow \epsilon$	$\langle A \rangle ::= \epsilon$	A se expande a la cadena vacía	

Ejemplo en BNF para sentencias condicionales:

`<sentencia> ::= <sentencia_asignacion>`

`| <sentencia_if>`

`| <sentencia_while>`

`| <sentencia_compuesta>`

`<sentencia_if> ::= "if" "(" <expresion> ")" <sentencia>`

`| "if" "(" <expresion> ")" <sentencia> "else" <sentencia>`

`<sentencia_compuesta> ::= "{" { <sentencia> } "}"`

Las llaves { } en la última línea indican cero o más repeticiones, una notación propia de EBNF (Extended BNF).

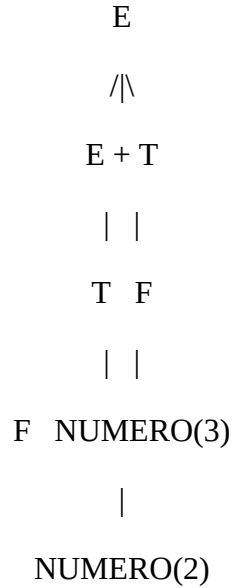
2.4.3. Árboles de Derivación y Árboles de Sintaxis Abstracta

El resultado del análisis sintáctico puede representarse mediante dos estructuras estrechamente relacionadas.

- **Árbol de derivación (parse tree)** : Es una representación gráfica completa de cómo una cadena de tokens ha sido generada a partir del símbolo inicial aplicando las reglas de producción. Cada nodo interno corresponde a un no terminal, y cada nodo hoja corresponde a un terminal (token). El árbol de derivación contiene toda la información gramatical, incluyendo símbolos no terminales que no añaden información semántica.
- **Árbol de sintaxis abstracta (Abstract Syntax Tree, AST)** : Es una versión simplificada y condensada del árbol de derivación, donde se omiten los no terminales que solo sirven para la estructura gramatical pero no aportan significado. Según Sebesta (2016), "el AST es la representación preferida para las fases posteriores del compilador (análisis semántico, optimización y generación de código) porque es más compacto y se centra en la estructura esencial del programa" (p. 287).

Ejemplo comparativo:

Para la expresión $2 + 3$, el árbol de derivación (según la gramática del apartado 2.4.1) sería:



El AST correspondiente, mucho más compacto, sería:



El AST elimina los no terminales T y F (que solo existían para fijar la precedencia de operadores) y retiene únicamente la operación y sus operandos.

2.4.4. Ambigüedad Sintáctica

Una gramática es ambigua si existe al menos una cadena que puede ser derivada mediante dos árboles de derivación distintos (es decir, que tiene dos interpretaciones estructurales diferentes). La ambigüedad es indeseable en lenguajes de programación porque un programa no debería tener más de un significado posible.

Ejemplo clásico de ambigüedad: La gramática que no respeta la precedencia de operadores.

Gramática ambigua para expresiones aritméticas:

$$E \rightarrow E + E \mid E * E \mid \text{NUMERO} \mid (E)$$

Para la expresión $2 + 3 * 4$, existen dos árboles de derivación posibles:

- Árbol 1: $(2 + 3) * 4 \rightarrow \text{resultado } 20$
- Árbol 2: $2 + (3 * 4) \rightarrow \text{resultado } 14$

Dado que la convención matemática establece que la multiplicación tiene mayor precedencia que la suma, el resultado correcto debería ser 14. La gramática ambigua no impone esta precedencia.

Resolución de la ambigüedad:

1. **Reescribir la gramática:** Introducir no terminales intermedios que impongan precedencia (como se hizo en la gramática no ambigua del apartado 2.4.1, con E, T, F).
2. **Especificar reglas adicionales:** Declarar explícitamente la precedencia y asociatividad de operadores fuera de la gramática, como se hace en generadores de analizadores como Yacc o Bison.

Según Ortega de la Puente et al. (2006), "la ambigüedad no es siempre un error en la definición de un lenguaje natural, pero en lenguajes de programación es inaceptable porque la computación debe ser determinista. Todo lenguaje de programación bien definido debe poseer una gramática no ambigua o, alternativamente, reglas de desambiguación explícitas" (p. 143).

2.4.5. Tipos de Analizadores Sintácticos

Existen dos familias principales de algoritmos de análisis sintáctico, cada una con sus ventajas y limitaciones.

Analizadores descendentes (top-down parsers) :

Estos analizadores comienzan desde el símbolo inicial e intentan expandirlo hasta generar la cadena de tokens de entrada. El tipo más conocido es el analizador predictivo recursivo descendente (y su variante tabular, LL(k)).

- **Ventajas:** Fáciles de implementar manualmente, producen mensajes de error razonablemente claros, se integran bien con el código de acción semántica.
- **Limitaciones:** Requieren gramáticas libres de recursión por la izquierda (no pueden tener reglas de la forma $A \rightarrow A \alpha$ porque causarían recursión infinita).
- **Aplicación:** Muchos compiladores educativos y comerciales usan analizadores descendentes escritos a mano (ej. el compilador de Python, el de GCC en sus primeras versiones).

Analizadores ascendentes (bottom-up parsers) :

Estos analizadores comienzan desde los tokens de entrada e intentan reducirlos (aplicando reglas de producción en sentido inverso) hasta alcanzar el símbolo inicial. El tipo más potente y utilizado es el analizador LR (Left-to-right, Rightmost derivation), especialmente sus variantes SLR, LALR y LR canónico.

- **Ventajas:** Pueden manejar una clase más amplia de gramáticas (incluyendo recursión por la izquierda), son más potentes que los LL(k), y los generadores automáticos (Yacc, Bison) producen analizadores muy eficientes.
- **Limitaciones:** Más difíciles de implementar manualmente; los mensajes de error tienden a ser menos claros que en los analizadores descendentes.
- **Aplicación:** La mayoría de los generadores de compiladores (Yacc, Bison, CUP, Racc) producen analizadores LALR. Muchos lenguajes reales (Java, C#, JavaScript en ciertas implementaciones) utilizan analizadores generados automáticamente.

Según Aho et al. (2007), "la mayoría de los lenguajes de programación modernos pueden ser descritos mediante gramáticas LALR(1), que constituyen un punto óptimo entre poder expresivo y eficiencia de análisis. El analizador LALR(1) típico tiene complejidad temporal $O(n)$ y espacial $O(n)$, donde n es la longitud del programa fuente" (p. 234).

2.4.6. Precedencia y Asociatividad de Operadores

Dos conceptos sintácticos fundamentales en la definición de lenguajes son la precedencia y la asociatividad.

Precedencia: Determina qué operador se evalúa primero cuando aparecen múltiples operadores en una expresión sin paréntesis. Por convención matemática universal:

- Exponenciación (mayor precedencia)
- Multiplicación y división (precedencia media)
- Suma y resta (menor precedencia)

En los lenguajes de programación, esta jerarquía se extiende con operadores relacionales ($<$, $>$, $=$), lógicos ($\&\&$, $\|\|$), de asignación ($=$), etc.

- **Asociatividad:** Determina el orden de evaluación cuando aparecen múltiples operadores de la misma precedencia.
- ✓ **Asociatividad izquierda:** $a - b - c$ se interpreta como $(a - b) - c$. Es la norma para la mayoría de operadores binarios.
- ✓ **Asociatividad derecha:** $a = b = c$ se interpreta como $a = (b = c)$. Es típica de la asignación y la exponenciación.

Cómo implementan precedencia y asociatividad los analizadores:

1. **En gramáticas no ambiguas:** Se introducen niveles de no terminales. Por ejemplo, la gramática del apartado 2.4.1 impone:
 - ✓ E (expresión) maneja $+$ (menor precedencia)

- ✓ T (término) maneja * (mayor precedencia)
- ✓ F (factor) maneja números y paréntesis (máxima precedencia)

2. **En generadores de analizadores (Yacc/Bison):** Se declaran explícitamente la precedencia y asociatividad mediante directivas como %left, %right, %nonassoc. Louden y Lambert (2012) observan: "La decisión de implementar la precedencia mediante la estructura de la gramática o mediante declaraciones externas es principalmente una cuestión de filosofía de diseño. Las gramáticas con muchos niveles de no terminales son más verbosas pero autónomas; las declaraciones externas son más compactas pero trasladan parte de la definición del lenguaje fuera de la gramática formal" (p. 256).

2.4.7. Manejo de Errores Sintácticos

Una funcionalidad crítica del analizador sintáctico es el manejo de errores. Ante una entrada sintácticamente incorrecta, el analizador debe:

1. **Detectar el error:** Identificar en qué punto la cadena de tokens deja de ajustarse a la gramática.
2. **Reportar el error:** Emitir un mensaje claro indicando la ubicación (número de línea y columna) y la naturaleza probable del error.
3. **Recuperarse del error:** Continuar el análisis para encontrar más errores en el mismo programa, sin detenerse en el primero.

Estrategias de recuperación de errores:

- **Modo pánico:** Se descartan tokens hasta encontrar un delimitador sincronizador (por ejemplo, un punto y coma o una llave de cierre) que permita reiniciar el análisis. Es simple y evita cascadas de errores falsos, pero puede omitir porciones significativas del código.
- **Recuperación a nivel de frase:** Se intenta corregir localmente el error insertando o eliminando tokens para que el resto del programa se ajuste a la gramática. Es más sofisticada pero compleja de implementar.
- **Producciones de error:** Se añaden reglas gramaticales especiales que capturan construcciones erróneas comunes, permitiendo emitir mensajes específicos.

Según Sebesta (2016), "la calidad del manejo de errores es una de las características que distingue a los compiladores profesionales de los educativos o experimentales. Un compilador que produce mensajes de error crípticos o que se detiene ante el primer error frustra al programador y ralentiza el desarrollo" (p. 324).

2.4.8. Herramientas para Generación de Analizadores Sintácticos

Al igual que en el análisis morfológico, existen herramientas que generan analizadores sintácticos automáticamente a partir de una especificación gramatical:

- **Yacc (Yet Another Compiler Compiler) y su versión GNU Bison:** Generan analizadores LALR(1) en C a partir de una gramática con acciones semánticas asociadas.
- **ANTLR (ANother Tool for Language Recognition):** Genera analizadores LL(*) (descendentes) en múltiples lenguajes (Java, C#, Python, Go, etc.). Es particularmente popular por su capacidad de generar analizadores legibles y por su soporte para gramáticas muy complejas.
- **JavaCC (Java Compiler Compiler):** Generador de analizadores descendentes para Java.
- **PLY (Python Lex-Yacc):** Reimplementación de Lex y Yacc en Python puro.

Estas herramientas permiten al diseñador de lenguajes centrarse en la especificación de alto nivel de la gramática, delegando la generación del código del analizador, la tabla de análisis y el manejo básico de errores.

2.4.9. Relación entre Sintaxis y Semántica

Es importante señalar que el análisis sintáctico verifica únicamente la *forma* del programa, no su significado. Un programa puede ser sintácticamente correcto pero semánticamente erróneo. Por ejemplo:

```
int x = "hola"; // Sintácticamente correcto (asignación)

// Semánticamente erróneo (tipo incompatible)
```

El análisis semántico (tercera fase del compilador) se encarga de estas comprobaciones: verificación de tipos, declaración de variables antes de su uso, control de flujo (funciones con return en todos los caminos), etc. El árbol de sintaxis abstracta producido por el analizador sintáctico es la estructura de datos sobre la que opera el analizador semántico.

Ortega de la Puente et al. (2006) resumen esta relación: "La sintaxis define el espacio de todos los programas posibles bien formados; la semántica restringe ese espacio a aquellos programas que además tienen significado dentro del modelo computacional del lenguaje. La separación entre ambas fases es una de las grandes conquistas de la ingeniería de compiladores" (p. 178).

Actividad I

1. Matriz Comparativa de Paradigmas de Programación

La siguiente tabla sintetiza los cinco paradigmas fundamentales abordados en esta investigación, destacando sus principios, ventajas, limitaciones y manifestación en lenguajes modernos (Sebesta, 2016).

Paradigma	Principios Fundamentales	Ventajas Clave	Limitaciones	Lenguajes Representativos
Imperativo / Estructural	Gestión explícita del estado, secuenciación de instrucciones, mutabilidad de memoria, efectos secundarios, estructuras de control básicas (bucles, condicionales)	Control preciso del hardware, eficiencia en sistemas embebidos, modelo mental sencillo	Complejidad en sistemas grandes, difícil mantenimiento, errores por estado mutable	C, Pascal, Zig
Orientado a Objetos (POO)	Encapsulamiento, polimorfismo, herencia vs composición, abstracción basada en datos y comportamiento combinados, envío de mensajes	Reutilización, modelado del mundo real, mantenibilidad, escalabilidad	Complejidad añadida, sobreingeniería, problemas con herencia múltiple	Java, C++, Python

Funcional	Inmutabilidad de datos, funciones como ciudadanas de primer orden, evaluación perezosa, transparencia referencial, ausencia de efectos colaterales	Código predecible, fácil prueba y depuración, concurrencia natural por diseño	Curva de aprendizaje pronunciada, overhead por inmutabilidad, no intuitivo para problemas con estado intensivo	Haskell, Elixir, Rust, JavaScript
Lógico / Declarativo	Programación basada en relaciones, unificación, cláusulas de Horn, abstracción total del flujo de control por parte del programador	Alto nivel de abstracción, demostración automática de teoremas, ideal para sistemas de reglas	Bajo rendimiento en problemas grandes, ecosistema reducido, poca aplicación comercial	Prolog, Datalog
Concurrente / Actores	Paso de mensajes entre entidades, aislamiento estricto de estado, mitigación de condiciones de carrera a nivel de diseño lingüístico, modelo "no share"	Concurrencia segura por diseño, ausencia de locks y deadlocks, escalabilidad horizontal	Overhead del paso de mensajes, depuración compleja, riesgo de desbordamiento de buzones	Erlang, Elixir, Rust (vía librerías)

2. Análisis Detallado por Paradigma

2.1 Paradigma Imperativo / Estructural

Fundamento teórico:

El paradigma imperativo concibe el programa como una secuencia de instrucciones que modifican explícitamente el estado del sistema. La memoria es mutable por defecto y los efectos secundarios (modificación de variables, entrada/salida) constituyen el mecanismo normal de operación (Sebesta, 2016). Este paradigma se basa directamente en el modelo de la máquina de von Neumann: memoria, unidad aritmético-lógica, unidad de control y contador de programa (Aho et al., 2008).

Caso de estudio: Zig como representante moderno

Zig fue seleccionado en esta investigación como exponente contemporáneo del enfoque imperativo estructurado por las siguientes razones (Equipo de Documentación de Zig, 2024):

- Ausencia de variables ocultas: No existe sobrecarga de operadores, ni preprocesador, ni flujo de control implícito (como excepciones no declaradas). Todo es explícito.
- Control directo de memoria: El programador debe gestionar explícitamente la memoria mediante un `allocator`, sin recolector de basura.
- Metaprogramación en tiempo de compilación: La palabra clave `comptime` permite ejecutar código en tiempo de compilación, eliminando overhead en tiempo de ejecución sin recurrir a macros complejas al estilo de C preprocesador.

Relación con compiladores:

Los compiladores de lenguajes imperativos (incluyendo Zig) realizan intensos análisis de flujo de datos para detectar variables no inicializadas, usos después de liberación y código inalcanzable (Aho et al., 2008). La optimización de saltos condicionales y la predicción de bifurcaciones son críticas para el rendimiento.

Ejemplo ilustrativo en Zig:

```
const std = @import("std");

pub fn main() !void {
    var x: i32 = 10;

    while (x > 0) {
        std.debug.print("{}\n", .{x});

        x -= 1; // Mutación explícita del estado
    } }
```

2.2 Paradigma Orientado a Objetos (POO)

Fundamento teórico:

La POO organiza el software como una colección de objetos discretos que combinan estado (atributos) y comportamiento (métodos). Los cuatro pilares clásicos son: encapsulamiento (ocultamiento de la representación interna), abstracción (modelado de entidades del dominio), herencia (reutilización mediante jerarquías) y polimorfismo (una interfaz, múltiples implementaciones) (Sebesta, 2016).

Debate contemporáneo: Herencia vs Composición

En la ingeniería de software moderna existe consenso en preferir la composición sobre la herencia (principio "Composition over Inheritance"). La herencia crea acoplamiento fuerte entre clases base y derivadas, mientras que la composición delega comportamiento a objetos independientes, incrementando la flexibilidad y el mantenimiento (Matsakis y Klock, 2014). Lenguajes como Rust han eliminado la herencia clásica en favor de traits (similares a interfaces con implementaciones por defecto), mientras que Python y JavaScript mantienen herencia pero promueven el uso de mixins y composición.

Convergencia multiparadigma en Python:

Python, uno de los cuatro lenguajes de trabajo de esta investigación, ejemplifica la convergencia multiparadigma al integrar POO con características funcionales: permite definir clases, herencia múltiple y polimorfismo, pero también incluye funciones lambda, map, filter, reduce y comprensiones de listas típicamente asociadas al paradigma funcional (Python Software Foundation, 2024).

Ejemplo ilustrativo en Python (composición preferida a herencia):

```
# Enfoque de composición (recomendado)

class Motor:

    def encender(self):

        return "Motor encendido"

class Vehiculo:

    def __init__(self):

        self.motor = Motor()    # Composición
```

```
def arrancar(self):
    return self.motor.encender()
```

2.3 Paradigma Funcional

Fundamento teórico:

La programación funcional trata la computación como evaluación de funciones matemáticas, evitando estado mutable y datos mutables. Se basa en el cálculo lambda (Church, 1930s) como fundamento formal (Sebesta, 2016). Cuatro conceptos son esenciales:

1. Inmutabilidad: Una vez que un símbolo recibe un valor, este nunca cambia. Las "actualizaciones" crean nuevos valores sin modificar los originales.
2. Funciones de primer orden: Las funciones pueden ser asignadas a variables, pasadas como argumentos y retornadas desde otras funciones (funciones de orden superior).

3.Caso de estudio: Rust y el estilo funcional

Rust, otro de los lenguajes de trabajo, incorpora un potente subsistema funcional basado en iteradores perezosos. Su sistema de ownership no entra en conflicto con la inmutabilidad; de hecho, las variables son inmutables por defecto (`let x = 5`), requiriendo `mut` explícito para mutabilidad (Fundación Rust, 2024).

Ejemplo ilustrativo en Rust (iteradores funcionales):

```
fn main() {
    let suma_pares_cuadrados: i32 = (1..=10)
        .filter(|x| x % 2 == 0)    // Evaluación perezosa
        .map(|x| x * x)           // Evaluación perezosa
        .sum();                   // Evaluación forzada aquí

    println!("{}", suma_pares_cuadrados);    // 220
    (4+16+36+64+100)
}
```

Ventaja en concurrencia: La transparencia referencial elimina por diseño las condiciones de carrera, porque no existe estado compartido mutable. Esta propiedad ha impulsado la adopción de elementos funcionales en lenguajes concurrentes (Armstrong, 2013).

2.4 Paradigma Lógico / Declarativo

Fundamento teórico:

A diferencia de los paradigmas anteriores, la programación lógica no especifica *cómo* resolver un problema, sino *qué* se quiere resolver. El programador declara hechos y reglas, y un motor de inferencia encuentra soluciones mediante unificación y backtracking (Sebesta, 2016).

Abstracción del flujo de control:

El programador no escribe bucles ni condicionales explícitos. La recursión y la búsqueda son gestionadas automáticamente por el intérprete lógico. Esto representa el nivel más alto de abstracción entre los paradigmas analizados (Van Roy y Haridi, 2004).

Limitaciones y aplicación actual:

Aunque el rendimiento es bajo para problemas de gran escala, la programación lógica encuentra aplicación en sistemas expertos, motores de reglas de negocio, procesamiento de grafos de conocimiento y verificación formal de software.

Ejemplo ilustrativo (Prolog conceptual):

```
% Hechos

genera_solar(100).    % Generación solar: 100 kW

demanda_industrial(80). % Demanda: 80 kW

% Regla de balance energético

balance_positivo :- genera_solar(X), demanda_industrial(Y), X > Y.

% Consulta

?- balance_positivo.  % Responde: true
```

2.5 Paradigma Concurrente / Actores

Fundamento teórico:

El modelo de actores (Hewitt, Bishop y Steiger, 1973) resuelve los problemas de la concurrencia clásica (locks, deadlocks, condiciones de carrera) mediante un diseño radicalmente diferente: los actores son entidades autónomas que poseen estado privado, se comunican exclusivamente mediante mensajes asíncronos, procesan un mensaje a la vez, y pueden crear otros actores (Armstrong, 2013).

Comparación con threading clásico:

Característica	Threads con memoria compartida	Modelo de Actores
Compartición de estado	Memoria compartida explícita	Ninguna (paso de mensajes)
Sincronización	Locks, semáforos, monitores	Por diseño (cola de mensajes)
Riesgos principales	Deadlocks, race conditions, livelocks	Overflow de buzón, mensajes huérfanos
Escalabilidad en multicore	Limitada por contención de locks	Alta (sin bloqueo global)

Convergencia multiparadigma en los lenguajes de trabajo:

- Rust: No tiene actores en la biblioteca estándar, pero ofrece canales (`std::sync::mpsc`) para paso de mensajes y librerías como `actix` que implementan el modelo completo (Fundación Rust, 2024).
- JavaScript: Web Workers + `postMessage` proporcionan aislamiento de estado y paso de mensajes, esencial para aplicaciones web con procesamiento en paralelo (Mozilla Developer Network, 2024).
- Python: El módulo `multiprocessing` con `Queue` implementa un estilo similar a actores aunque con mayor overhead (Python Software Foundation, 2024).

Ejemplo ilustrativo conceptual (JavaScript con Web Worker):

```
// worker.js

self.onmessage = function(event) {

    let resultado = procesar(event.data);

    self.postMessage(resultado);

};

// main.js
```

```
const worker = new Worker('worker.js');

worker.postMessage(datos);

worker.onmessage = function(event) {

    console.log('Resultado:', event.data);

};
```

3. Convergencia Multiparadigma en Lenguajes Modernos

El fenómeno más relevante en la evolución de los lenguajes de programación es la hibridación multiparadigma (Sebesta, 2016). Los cuatro lenguajes seleccionados para esta investigación evidencian esta tendencia:

Lenguaje	Paradigma Principal	Paradigmas Integrados	Justificación de diseño
Zig	Imperativo / Estructural	Funcional (parcial con <code>comptime</code>), metaprogramación	Reemplazar a C con control explícito pero añadiendo capacidades de metaprogramación modernas sin macros (Equipo de Documentación de Zig, 2024).
Python	Multiparadigma (OO + Imperativo)	Funcional (lambda, map, filter, comprehensions)	Versatilidad para ciencia de datos, scripting y desarrollo web; sacrifica rendimiento por legibilidad (Python Software Foundation, 2024).

Rust	Multiparadigma (Funcional + Imperativo)	Traits (similar a OO sin herencia), paso de mensajes (canales)	Seguridad de memoria sin GC + alto rendimiento; toma lo mejor de funcional (inmutabilidad) e imperativo (control fino) (Matsakis y Klock, 2014; Fundación Rust, 2024).
JavaScript	Multiparadigma (Prototípico + Funcional)	Imperativo, eventos, asíncrono (no n-blocking)	Originalmente scripting web; evolucionó para servidores (Node.js) con modelo asíncrono y estilo funcional (Mozilla Developer Network, 2024).

Análisis crítico de la convergencia

La industria del software ha abandonado el dogma de la pureza paradigmática por razones prácticas (Sebesta, 2016):

1. Demandas del mercado: Un lenguaje debe ser competente en múltiples dominios: desarrollo web, sistemas embebidos, ciencia de datos, servidores de alto rendimiento.
2. Curva de aprendizaje suave: Los equipos de desarrollo ya conocen POO e imperativo. Introducir gradualmente características funcionales es más viable que migrar a lenguajes puramente funcionales.
3. Rendimiento vs Abstracción: El programador moderno necesita alto nivel para productividad y bajo nivel para optimización. Rust es un ejemplo logrado de esta síntesis (Matsakis y Klock, 2014).
4. Concurrencia obligatoria: Con el estancamiento de la ley de Moore y el auge de los procesadores multicore, los modelos de paso de mensajes se integran en lenguajes previamente imperativos (Armstrong, 2013).

Actividad II

1. Algoritmo seleccionado

Para el estudio de benchmarking se seleccionó la Conjetura de Collatz (también conocida como problema $3n+1$). Este algoritmo fue elegido por las siguientes razones:

- Genera carga de procesamiento intensiva, para comparar rendimiento de lenguajes.
- Su lógica es simple (solo divisiones y multiplicaciones), lo que permite implementarlo fácilmente en los cuatro lenguajes sin depender de bibliotecas externas.
- Permite calcular una métrica objetiva (total de pasos acumulados) que debe ser idéntica en todos los lenguajes, validando así la correcta implementación.

El algoritmo se ejecutó para todos los números desde 1 hasta $N = 100,000$, acumulando el total de pasos de cada número. El valor esperado del total de pasos es 10.753.840, fue verificado en los cuatro lenguajes, confirmando la correcta implementación.

2. Análisis Morfológicos

El análisis morfológico es la primera fase del proceso de compilación, encargada de convertir el código fuente en una secuencia de tokens. A continuación se detalla esta estructura léxica para los cuatro lenguajes de trabajo: Zig, Python, JavaScript y Rust.

2.1. Análisis Morfológico del Lenguaje Zig

Zig es un lenguaje compilado de sistemas diseñado como sucesor moderno de C, con control explícito de memoria y metaprogramación en tiempo de compilación. Su estructura léxica es simple pero rigurosa, lo que facilita el análisis por parte del compilador (Equipo de Documentación de Zig, 2024). A continuación se detallan todos los componentes morfológicos que el analizador léxico debe reconocer.

Aspecto Morfológico	Descripción Completa
Palabras reservadas	Zig cuenta con 37 palabras reservadas que no pueden ser utilizadas como identificadores por el programador. La lista completa es: align, allowzero, and, anyframe, anytype, asm, await, break, callconv, catch, comptime, const, continue, defer, else, enum, errdefer, error, export, extern, fn, for, if, inline, noalias, nosuspend, null, or, orelse, pub, resume, return, struct, suspend, switch, test, threadlocal, try, union, unreachable, usin namespace, var, while, volatile. Estas palabras tienen propósitos específicos: const declara constantes inmutables; var declara variables mutables; fn define funciones; pub controla la visibilidad pública de un símbolo; comptime ejecuta código en tiempo de compilación; try y catch gestionan errores; defer y errdefer posponen ejecución al salir del ámbito; struct, enum, union definen tipos compuestos; if, else, while, for, switch controlan el flujo; break y continue alteran bucles; return sale de una función; suspend y resume gestionan funciones asíncronas; asm inserta código ensamblador en línea; test define pruebas unitarias; threadlocal declara variables con almacenamiento local por hilo.
Identificadores (reglas de formación)	Los identificadores son nombres que el programador asigna a variables, funciones, tipos y otros elementos. Las reglas de Zig son: (1) deben comenzar con una letra minúscula (a-z), letra mayúscula (A-Z) o guion bajo (_): (2) después del primer carácter, pueden contener letras minúsculas, letras mayúsculas, números (0-9) o guiones bajos (_); (3) no pueden comenzar con un número, para evitar ambigüedad con los literales numéricos; (4) son sensibles a mayúsculas y minúsculas (por lo que contador, Contador y CONTADOR son identificadores diferentes); (5) no pueden coincidir con ninguna de las 37 palabras reservadas. Ejemplos válidos: contador, _temp, MAXIMO, valorInicial123. Ejemplos inválidos: 123var (comienza con número), if (palabra reservada).
Literales numéricos	Zig soporta múltiples representaciones de números: (1) Enteros decimales: secuencia de dígitos 0-9, ej. 123, 0, 42. (2) Enteros hexadecimales: prefijo 0x seguido de dígitos 0-9 y letras A-F (mayúsculas o minúsculas), ej. 0x7B (123 decimal). (3) Enteros octales: prefijo 0o seguido de dígitos 0-7, ej. 0o173 (123 decimal). (4) Enteros binarios: prefijo 0b seguido de dígitos 0-1, ej. 0b1111011 (123 decimal). (5) Separadores visuales: el guion bajo _ puede insertarse entre dígitos para mejorar la

	legibilidad sin afectar el valor, ej. 1_000_000, 0b1111_0000. (6) Números reales (comma flotante): incluyen un punto decimal, ej. 3.1416, 0.5, 123.0. También soportan notación científica con e o E, ej. 1.2e-5, 3E10. Los reales pueden usar separadores visuales, ej. 3.1415_9265.
Literales de caracteres	Un literal de carácter representa un único carácter Unicode. Se escribe entre comillas simples (<code>'</code>). Soporta: (1) Caracteres normales: <code>'a', 'Z', 'ñ', 'ç'</code>. (2) Secuencias de escape estándar: <code>\n</code> (nueva línea), <code>\r</code> (retorno de carro), <code>\t</code> (tabulador horizontal), <code>\\</code> (barra invertida), <code>'</code> (comilla simple), <code>\"</code> (comilla doble). (3) Secuencias de escape hexadecimales: <code>\x</code> seguido de dos dígitos hexadecimales, ej. <code>\x7F</code>. (4) Secuencias de escape Unicode: <code>\u{</code> seguido de uno a seis dígitos hexadecimales y <code>}</code>, ej. <code>\u{1F600}</code> (emoji cara sonriente).
Literales de cadena	Una cadena es una secuencia de caracteres. Se escribe entre comillas dobles (<code>"</code>). Soporta las mismas secuencias de escape que los literales de carácter. Las cadenas pueden ser: (1) De una línea: <code>"Hola, mundo"</code>, <code>"Línea\ncon salto"</code>. (2) Multilínea: usando <code>\\</code> al final de la línea para indicar que el salto de línea no debe incluirse en la cadena. Ejemplo: <code>"Esta línea continúa\\ en la siguiente línea"</code>.
Comentarios (tipos completos)	Zig diferencia cuatro tipos de comentarios, todos ignorados por el compilador: (1) Comentarios de línea: comienzan con <code>//</code> y continúan hasta el final de la línea. Ejemplo: <code>// Esto es un comentario</code>. (2) Comentarios de bloque: comienzan con <code>/*</code> y terminan con <code>*/</code>. Pueden abarcar múltiples líneas. No son anidables (un <code>/*</code> dentro de otro no es válido). Ejemplo: <code>/* Esto es un comentario de bloque */</code>. (3) Comentarios de documentación pública: comienzan con <code>///</code> (tres barras). Se utilizan para documentar elementos públicos (funciones, tipos, constantes) y son procesados por herramientas de generación de documentación. Ejemplo: <code>/// Esta función calcula la raíz cuadrada</code>. (4) Comentarios de documentación de módulo: comienzan con <code>//!</code> (dos barras y un signo de exclamación). Se colocan al inicio del archivo para documentar el módulo completo. Ejemplo: <code>//! Módulo de operaciones matemáticas</code>.
Delimitadores de bloques	Zig utiliza llaves { ... } para delimitar bloques de código. Todo bloque de sentencias (cuerpo de una función, bloque de un if, cuerpo de un while, etc.) debe estar encerrado entre llaves. La indentación no tiene significado sintáctico: los espacios o tabuladores al inicio de línea son ignorados por el analizador léxico. Esto diferencia a Zig de lenguajes como Python, donde la indentación es obligatoria y define la estructura del bloque. En Zig, el siguiente código es válido aunque la indentación sea inconsistente: <code>if (x > 0) { std.debug.print("ok"); }</code>.

Elementos irrelevantes (espacios y delimitadores)	El analizador léxico de Zig ignora completamente los siguientes caracteres, excepto cuando sirven para separar tokens: (1) espacios en blanco (); (2) tabuladores horizontales (\t); (3) tabuladores verticales; (4) retornos de carro (\r); (5) saltos de línea (\n). Por lo tanto, <code>if(x>0){print("ok");}</code> es equivalente a <code>if (x > 0) { print("ok"); }</code>. El analizador también ignora los comentarios (todos los tipos).
Características especiales (unicidad de Zig)	Zig tiene dos características morfológicas notables frente a otros lenguajes: (1) Ausencia de preprocesador: no existen directivas como <code>#define</code> o <code>#include</code> (propias de C). La metaprogramación se realiza con comptime dentro del propio lenguaje. (2) No sobrecarga de operadores: los operadores (+, -, *, etc.) tienen un significado fijo y no pueden ser redefinidos por el programador. (3) Sin macros textuales: no hay macros que operen sobre el texto fuente; la generación de código en tiempo de compilación se hace con comptime.

2.2. Análisis Morfológico del Lenguaje Python

Python es un lenguaje interpretado de alto nivel, conocido por su sintaxis clara y legible. Su característica morfológica más distintiva es la indentación significativa para delimitar bloques, lo que lo diferencia de lenguajes como Zig, Rust o JavaScript que usan llaves (Python Software Foundation, 2024). A continuación se detallan todos los componentes morfológicos que el analizador léxico de Python debe reconocer.

Aspecto Morfológico	Descripción Completa
Palabras reservadas (lista completa)	Python 3.x cuenta con 35 palabras reservadas que no pueden ser utilizadas como identificadores por el programador. La lista completa es: False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield. Estas palabras tienen propósitos específicos: def define funciones; class define clases; if, elif, else control condicional; for, while bucles; break, continue alteran bucles; return sale de una función devolviendo un valor; yield convierte una función en generador; lambda define funciones anónimas; import, from, as gestionan

	módulos; try, except, finally, raise manejan excepciones; with gestiona contextos (ej. apertura de archivos); assert verifica condiciones en tiempo de desarrollo; pass es una operación nula (no hace nada); global y nonlocal modifican el ámbito de las variables; and, or, not son operadores lógicos; in, is son operadores de comparación; await, async soportan programación asíncrona; del elimina referencias a objetos; False, True, None son valores literales booleanos y nulo.
Identificadores (reglas de formación)	Los identificadores son nombres que el programador asigna a variables, funciones, clases, módulos y otros elementos. Las reglas de Python son: (1) deben comenzar con una letra (a-z, A-Z) o guion bajo (_); (2) después del primer carácter, pueden contener letras minúsculas, letras mayúsculas, números (0-9) o guiones bajos (_); (3) no pueden comenzar con un número, para evitar ambigüedad con los literales numéricos; (4) son sensibles a mayúsculas y minúsculas (por lo que contador, Contador y CONTADOR son identificadores diferentes); (5) no pueden coincidir con ninguna de las 35 palabras reservadas. Además, Python tiene convenciones especiales para identificadores con guiones bajos: _nombre (convención para atributo "protegido" o interno), nombre_ (para evitar conflictos con palabras reservadas), __nombre (mangling para atributos privados de clase), __nombre__ (nombres "mágicos" o especiales definidos por el lenguaje, como __init__, __str__, __len__). Ejemplos válidos: contador, _temp, MAXIMO, valorInicial123, __init__. Ejemplos inválidos: 123var (comienza con número), if (palabra reservada), mi-variable (contiene guion medio, que no está permitido porque se interpretaría como resta).
Literales numéricos	Python soporta múltiples representaciones de números: (1) Enteros decimales: secuencia de dígitos 0-9, ej. 123, 0, 42, 1000000. (2) Enteros hexadecimales: prefijo 0x seguido de dígitos 0-9 y letras A-F (mayúsculas o minúsculas), ej. 0x7B (123 decimal), 0xFF (255 decimal). (3) Enteros octales: prefijo 0o seguido de dígitos 0-7, ej. 0o173 (123 decimal). (4) Enteros binarios: prefijo 0b seguido de dígitos 0-1, ej. 0b1111011 (123 decimal). (5) Separadores visuales: el guion bajo _ puede insertarse entre dígitos para mejorar la legibilidad sin afectar el valor, ej. 1_000_000, 0b1111_0000, 0xDEAD_BEEF. (6) Números enteros de precisión arbitraria: Python no tiene límite superior para enteros (big integers), ej. 10**100 representa un googol. (7) Números reales (coma flotante): incluyen un punto decimal, ej. 3.1416, 0.5, 123.0. También soportan notación científica con e o E, ej. 1.2e-5, 3E10. (8) Números complejos: se representan con la unidad imaginaria j, ej. 3+4j, 1.5j.

Literales de caracteres	Python no tiene un tipo carácter distinto del tipo cadena. Un carácter es simplemente una cadena de longitud 1. Por lo tanto, los literales de carácter se escriben como cadenas: entre comillas simples ('a') o comillas dobles ("b"). Soporta las mismas secuencias de escape que las cadenas.
Literales de cadena	Python ofrece múltiples formas de representar cadenas, todas soportan secuencias de escape como <code>\n</code> (nueva línea), <code>\t</code> (tabulador), <code>\\</code> (barra invertida), <code>\'</code> (comilla simple), <code>\"</code> (comilla doble), <code>\xNN</code> (byte hexadecimal), <code>\uNNNN</code> (Unicode 16 bits), <code>\UNNNNNNNN</code> (Unicode 32 bits). Los tipos de cadenas son: (1) Comillas simples: <code>'Hola mundo'</code>. (2) Comillas dobles: <code>"Hola mundo"</code>. (3) Comillas triples simples: <code>"""Texto multilínea\ncon saltos de línea explícitos"""</code>. (4) Comillas triples dobles: <code>"""\"Texto multilínea\ncon saltos de línea explícitos\""""</code>. Las comillas triples permiten cadenas que abarcan múltiples líneas; los saltos de línea se incluyen en la cadena. (5) Cadenas prefijadas: <code>f"Suma: {a+b}"</code> (f-string, permite interpolación de variables); <code>r"C:\archivo\no\escapa"</code> (raw string, las barras invertidas no se interpretan como escapes); <code>b"datos binarios"</code> (bytes, no cadena Unicode); <code>u"texto Unicode"</code> (obsoleto en Python 3, las cadenas son Unicode por defecto).
Comentarios (tipos completos)	Python solo tiene comentarios de línea. Todo lo que sigue al carácter <code>#</code> (fuera de una cadena literal) hasta el final de la línea es ignorado por el intérprete. No existen comentarios de bloque multilínea como <code>/* ... */</code> en otros lenguajes. Sin embargo, se puede simular un comentario de bloque usando cadenas multilínea no asignadas (con comillas triples), aunque esto no es una práctica recomendada porque el intérprete sí procesa la cadena (aunque no hace nada con ella). Ejemplos: <code># Esto es un comentario de línea</code>. También se utilizan cadenas de documentación (docstrings) con comillas triples al inicio de módulos, funciones o clases para documentación, pero no son comentarios ignorados por el intérprete; son accesibles en tiempo de ejecución mediante <code>obj.__doc__</code>. Ejemplo de docstring: <code>def mi_funcion(): """Esta función hace algo.""" pass</code>.
Delimitadores de bloques (característica distintiva)	Característica morfológica más distintiva de Python: la indentación (espacios o tabuladores al inicio de línea) tiene significado sintáctico y es la única forma de delimitar bloques de código. No se utilizan llaves { } ni palabras clave como <code>end</code> (como en Ruby) para cerrar bloques. Las reglas son: (1) Cada bloque indentado define un ámbito; (2) La indentación debe ser consistente dentro del mismo bloque (se pueden usar espacios o tabuladores, pero no mezclarlos); (3) Por convención (recomendada por PEP 8), se usan 4 espacios por nivel de indentación; (4) El final de la indentación (desindentación) marca el fin del bloque. Un ejemplo de bloque en Python: <code>if x > 0:\n print("positivo")\n print("y mayor que cero")\nprint("esto</code>

	está fuera del if"). Este diseño fuerza un código más legible y estructurado, pero es una fuente de errores si se mezclan espacios y tabuladores.
Elementos irrelevantes	El analizador léxico de Python ignora los siguientes caracteres excepto cuando sirven para separar tokens o para definir indentación: (1) espacios en blanco () que no están al inicio de línea (los que están al inicio de línea definen indentación, por lo que no son irrelevantes); (2) tabuladores horizontales (\t) al inicio de línea (también definen indentación); (3) retornos de carro (\r) y saltos de línea (\n) que no forman parte de la indentación; (4) espacios y tabuladores que no están al inicio de línea, después de un token, son ignorados (ej. a+b es equivalente a a + b). Diferencia fundamental con Zig, Rust y JavaScript: en Python, los espacios al inicio de línea no pueden ser ignorados porque definen la estructura del programa. Un programa con indentación inconsistente produce un error de sintaxis (IndentationError).
Características especiales (unicidad de Python)	Python tiene varias características morfológicas notables frente a otros lenguajes: (1) Indentación significativa: única forma de delimitar bloques, sin llaves ni palabras clave de cierre. (2) Ausencia de punto y coma: el punto y coma (;) es opcional como separador de sentencias en la misma línea, pero no es obligatorio al final de cada línea como en C, Java o Rust. (3) Comillas triples: permiten cadenas multilínea sin necesidad de escapes. (4) Raw strings (r""): las barras invertidas no se interpretan como escapes, útil para rutas de Windows o expresiones regulares. (5) F-strings (f""): permiten interpolación de variables con sintaxis {variable}. (6) No tiene comentarios de bloque: solo comentarios de línea con #.

2.3. Análisis Morfológico del Lenguaje JavaScript (Node.js)

JavaScript es un lenguaje de programación dinámico, basado en prototipos y eventos, que se ejecuta principalmente en motores de alto rendimiento como V8 (Chrome, Node.js) o SpiderMonkey (Firefox). Originalmente diseñado para la web, ha expandido su propósito a servidores, herramientas de línea de comandos y sistemas embebidos. Su estructura léxica está estandarizada por ECMAScript (Mozilla Developer Network, 2024). A continuación se detallan todos los componentes morfológicos que el analizador léxico de JavaScript debe reconocer.

Aspecto Morfológico	Descripción Completa
Palabras reservadas	JavaScript (ECMAScript 2024) cuenta con un conjunto de palabras reservadas que no pueden ser utilizadas como identificadores por el programador. Se dividen en varias categorías: (1) Palabras clave del lenguaje: break, case, catch, class, const, continue, debugger, default, delete, do, else, export, e

	xtends, finally, for, function, if, import, in, instanceof, new, return, super, switch, this, throw, try, typeof, var, void, while, with, yield. (2) Palabras reservadas para uso futuro en modo estricto: implements, interface, let, package, private, protected, public, static. (3) Palabras reservadas para uso futuro en módulos: await. (4) Literales reservados: true, false, null. Estas palabras tienen propósitos específicos: function define funciones; class define clases; const declara constantes (no reasignables); let y var declaran variables (con diferencias de ámbito); if, else control condicional; for, while, do bucles; break, continue alteran bucles; return sale de una función devolviendo un valor; yield detiene y reanuda funciones generadoras; await pausa funciones asíncronas; try, catch, finally, throw manejan excepciones; switch, case, default control múltiple; import, export gestionan módulos; class, extends, super soportan programación orientada a objetos basada en clases; new crea instancias de objetos; this referencia al contexto de ejecución; typeof devuelve el tipo de un operando; instanceof verifica la cadena de prototipos; delete elimina propiedades de objetos; in verifica existencia de propiedades; void evalúa una expresión y devuelve undefined; debugger invoca depuración.
Identificadores (reglas de formación)	Los identificadores son nombres que el programador asigna a variables, funciones, clases, parámetros y otros elementos. Las reglas de JavaScript son: (1) deben comenzar con una letra (a-z, A-Z), el signo de dólar (\$) o el guion bajo (_); (2) después del primer carácter, pueden contener letras minúsculas, letras mayúsculas, números (0-9), el signo de dólar (\$) o guiones bajos (_); (3) pueden contener caracteres Unicode (como letras acentuadas, caracteres griegos o emojis, aunque no es práctica común); (4) no pueden comenzar con un número, para evitar ambigüedad con los literales numéricos; (5) son sensibles a mayúsculas y minúsculas (por lo que contador, Contador y CONTADOR son identificadores diferentes); (6) no pueden coincidir con ninguna de las palabras reservadas enumeradas anteriormente. Convenciones comunes: identificadores que comienzan con _ (ej. _temp) se usan convencionalmente para indicar "privado" o "interno", aunque el lenguaje no lo impone; el signo de dólar \$ era usado por bibliotecas como jQuery (ej. \$ o \$div) pero no tiene significado especial en el lenguaje. Ejemplos válidos: contador, \$temp, _valor, π (Unicode), miVariable123. Ejemplos inválidos: 123var (comienza con número), if (palabra reservada), mi-variable (contiene guion medio, que se interpretaría como resta).

Literales numéricos	JavaScript soporta múltiples representaciones de números, todos ellos de tipo Number (coma flotante de doble precisión IEEE 754) y BigInt (enteros de precisión arbitraria). (1) Enteros decimales: secuencia de dígitos 0-9, ej. 123, 0, 42, 1000000. (2) Enteros hexadecimales: prefijo 0x seguido de dígitos 0-9 y letras A-F (mayúsculas o minúsculas), ej. 0x7B (123 decimal), 0xFF (255 decimal). (3) Enteros octales: en modo estricto, prefijo 0o seguido de dígitos 0-7, ej. 0o173 (123 decimal). (4) Enteros binarios: prefijo 0b seguido de dígitos 0-1, ej. 0b1111011 (123 decimal). (5) Separadores visuales: el guion bajo _ puede insertarse entre dígitos para mejorar la legibilidad sin afectar el valor, ej. 1_000_000, 0b1111_0000, 0xDEAD_BEEF. (6) Números reales (coma flotante): incluyen un punto decimal, ej. 3.1416, 0.5, 123.0. También soportan notación científica con e o E, ej. 1.2e-5, 3E10. (7) BigInt: se crea añadiendo el sufijo n al final de un literal entero, ej. 123n, 0x7Bn, 1_000_000n. Permite enteros de cualquier tamaño sin pérdida de precisión. (8) Valores numéricos especiales: Infinity (infinito), -Infinity (-infinito), NaN (Not-a-Number, resultado de operaciones inválidas).
Literales de caracteres	JavaScript no tiene un tipo carácter distinto del tipo cadena. Un carácter es simplemente una cadena de longitud 1. Por lo tanto, los literales de carácter se escriben como cadenas: entre comillas simples ('a') o comillas dobles ("b"). También pueden usarse backticks para plantillas (template literals). Soporta las mismas secuencias de escape que las cadenas.
Literales de cadena	JavaScript ofrece múltiples formas de representar cadenas, todas soportan secuencias de escape como \n (nueva línea), \t (tabulador), \\ (barra invertida), \' (comilla simple), \" (comilla doble), \xNN (byte hexadecimal), \uNNNN (Unicode 16 bits), \u{NNNNN} (Unicode con notación extendida). Los tipos de cadenas son: (1) Comillas simples: 'Hola mundo'. (2) Comillas dobles: "Hola mundo". (3) Template literals (backticks): `Hola mundo`. Los template literals permiten interpolación de variables con la sintaxis \${variable} y pueden abarcar múltiples líneas sin necesidad de escapes. Ejemplo: `Hola \${nombre}\n¿Cómo estás?`. (4) Raw strings: no existen como prefijo nativo, pero se puede simular con funciones o usando barras invertidas escapadas.

Comentarios (tipos completos)	JavaScript soporta dos tipos de comentarios, ambos ignorados por el motor de ejecución: (1) Comentarios de línea: comienzan con <code>//</code> y continúan hasta el final de la línea. Ejemplo: <code>// Esto es un comentario.</code> (2) Comentarios de bloque: comienzan con <code>/*</code> y terminan con <code>*/</code>. Pueden abarcar múltiples líneas. Pueden ser anidados en algunos motores, pero no es estándar seguro. Ejemplo: <code>/* Esto es un comentario de bloque */</code>. (3) Comentarios de documentación (JSDoc): comienzan con <code>/**</code> y terminan con <code>*/</code>. Son utilizados por herramientas de generación de documentación y editores para proporcionar información de tipos. Ejemplo: <code>/** @param {number} x - Descripción */</code>.
Delimitadores de bloques	JavaScript utiliza llaves <code>{ ... }</code> para delimitar bloques de código en funciones, condicionales (<code>if</code>, <code>else</code>), bucles (<code>for</code>, <code>while</code>, <code>do</code>), <code>switch</code>, <code>try/catch/finally</code>, y <code>class</code>. La indentación no tiene significado sintáctico: los espacios o tabuladores al inicio de línea son ignorados por el analizador léxico. Esto diferencia a JavaScript de lenguajes como Python. El uso de llaves es obligatorio para bloques con múltiples sentencias, pero opcional para bloques de una sola sentencia (aunque se recomienda usarlas siempre por legibilidad y para evitar errores). Ejemplo: <code>if (x > 0) { console.log("positivo"); }</code>. Una sentencia sin llaves: <code>if (x > 0) console.log("positivo");</code> (válido, pero la sentencia siguiente ya no pertenece al <code>if</code>).
Elementos irrelevantes	El analizador léxico de JavaScript ignora los siguientes caracteres excepto cuando sirven para separar tokens: (1) espacios en blanco <code>()</code>; (2) tabuladores horizontales <code>(\t)</code>; (3) retornos de carro <code>(\r)</code>; (4) saltos de línea <code>(\n)</code>. Por lo tanto, <code>if(x>0){console.log("ok");}</code> es equivalente a <code>if (x > 0) { console.log("ok"); }</code>. A diferencia de Python, los espacios al inicio de línea no tienen significado sintáctico y son completamente ignorados. También se ignoran los comentarios (todos los tipos). El punto y coma <code>(;)</code> es opcional como terminador de sentencia debido a la inserción automática de punto y coma (ASI, Automatic Semicolon Insertion), aunque se recomienda usarlo explícitamente para evitar comportamientos inesperados.
Características especiales (unicidad de JavaScript)	JavaScript tiene varias características morfológicas notables frente a otros lenguajes: (1) Sensibilidad a mayúsculas/minúsculas estricta: Variable y variable son diferentes. (2) Uso de \$ y _: permitidos como caracteres iniciales de identificadores, no solo letras. (3) Template literals con backticks (`): permiten interpolación de variables

	con <code>\${}</code> y cadenas multilínea sin escapes. (4) Inserción automática de punto y coma (ASI): el motor inserta ; automáticamente al final de las líneas donde sea necesario, lo que hace que el punto y coma sea opcional en la mayoría de los casos. (5) Comentarios de bloque /* */: pueden contener cualquier carácter excepto */, y pueden extenderse por múltiples líneas. (6) Números especiales: NaN, Infinity, -Infinity son literales numéricos válidos. (7) BigInt con sufijo n: enteros de precisión arbitraria.
--	--

2.4. Análisis Morfológico del Lenguaje Rust

Rust es un lenguaje compilado de sistemas diseñado para ofrecer seguridad de memoria sin necesidad de un recolector de basura, mediante un sistema de ownership, borrowing y lifetimes. Su sintaxis está influenciada por C++ pero con influencias funcionales de lenguajes como Haskell y OCaml (Fundación Rust, 2024). A continuación se detallan todos los componentes morfológicos que el analizador léxico de Rust debe reconocer.

Aspecto Morfológico	Descripción Completa
Palabras reservadas (lista completa)	Rust cuenta con un conjunto de palabras reservadas que no pueden ser utilizadas como identificadores por el programador. Se dividen en tres categorías según la documentación oficial (Rust Reference): (1) Palabras clave estrictas (siempre reservadas): as, break, const, continue, crate, dyn, else, enum, extern, false, fn, for, if, impl, in, let, loop, match, mod, move, mut, pub, ref, return, self, Self, static, struct, super, trait, true, type, unsafe, use, where, while. (2) Palabras clave reservadas para uso futuro (posiblemente se añadan en versiones posteriores) : abstract, become, box, do, final, macro, override, priv, try, typeof, unsized, virtual, yield. (3) Palabras clave que no pueden usarse como identificadores en el ámbito de atributos y macros: raw. Estas palabras tienen propósitos específicos: fn define funciones; let declara variables; mut indica mutabilidad; const declara constantes en tiempo de compilación; static declara variables estáticas; pub controla visibilidad pública; mod define módulos; use importa elementos; struct define estructuras; enum define enumeraciones; trait define interfaces; impl implementa funcionalidad; match realiza pattern matching; if, else control condicional; loop, while, for bucles; break, continue alteran bucles; return sale de una función; unsafe permite operaciones no seguras; dyn indica objetos trait dinámicos; move transfiere ownership; ref crea referencias; Self alias del tipo

	implementado; self parámetro receptor de método; super módulo padre; crate módulo raíz; as realiza conversiones; in en bucles for; extern llama a código externo (FFI); type define alias de tipos; where restringe parámetros genéricos; true y false literales booleanos.
Identificadores (reglas de formación)	Los identificadores son nombres que el programador asigna a variables, funciones, tipos, constantes, módulos y otros elementos. Las reglas de Rust son: (1) deben comenzar con una letra minúscula (a-z), letra mayúscula (A-Z) o guion bajo (_); (2) después del primer carácter, pueden contener letras minúsculas, letras mayúsculas, números (0-9) o guiones bajos (_); (3) pueden contener caracteres Unicode (como letras acentuadas, caracteres griegos, etc.), siempre que sean caracteres de la categoría Unicode XID_Start (inicio) y XID_Continue (continuación); (4) no pueden comenzar con un número, para evitar ambigüedad con los literales numéricos; (5) son sensibles a mayúsculas y minúsculas (por lo que contador, Contador y CONTADOR son identificadores diferentes); (6) no pueden coincidir con ninguna de las palabras reservadas estrictas, pero pueden coincidir con las reservadas para uso futuro (aunque se desaconseja). Convenciones de Rust (estilo oficial): (a) variables, funciones y módulos usan snake_case (ej. mi_variable, calcular_suma); (b) tipos (structs, enums, traits) usan PascalCase (ej. MiStruct, Color); (c) constantes y statics usan SCREAMING_SNAKE_CASE (ej. MAXIMO, VALOR_PI); (d) identificadores con un guion bajo inicial (_) (ej. _ignorado) se usan para valores no utilizados (el compilador no emite advertencias por no usarlos); (e) identificadores con doble guion bajo __ tienen significados especiales (reservados por el compilador). Ejemplos Validos: contador, _temp, MAXIMO, valor_inicial_123, MiStruct, Π (Unicode). Ejemplos inválidos: 123var (comienza con número), if (palabra reservada estricta), mi-variable (contiene guion medio, no permitido).
Literales numéricos	Rust soporta múltiples representaciones de números, con tipos enteros y de coma flotante explícitos: (1) Enteros decimales: secuencia de dígitos 0-9, ej. 123, 0, 42, 1000000. (2) Enteros hexadecimales: prefijo 0x seguido de dígitos 0-9 y letras A-F (mayúsculas o minúsculas), ej. 0x7B (123 decimal), 0xFF (255 decimal).

	(3) Enteros octales: prefijo 0o seguido de dígitos 0-7, ej. 0o173 (123 decimal). (4) Enteros binarios: prefijo 0b seguido de dígitos 0-1, ej. 0b1111011 (123 decimal). (5) Separadores visuales: el guion bajo _ puede insertarse entre dígitos para mejorar la legibilidad sin afectar el valor, ej. 1_000_000, 0b1111_0000, 0xDEAD_BEEF. (6) Sufijos de tipo para enteros: se puede añadir un sufijo para especificar el tipo exacto, ej. 123i8 (entero con signo de 8 bits), 123u16 (entero sin signo de 16 bits), 123i32 (32 bits por defecto), 123u64 (64 bits), 123isize (tamaño de puntero), 123usize (tamaño de puntero sin signo). (7) Números reales (coma flotante): incluyen un punto decimal, ej. 3.1416, 0.5, 123.0. También soportan notación científica con e o E, ej. 1.2e-5, 3E10. (8) Sufijos de tipo para reales: 3.14f32 (float de 32 bits), 3.14f64 (float de 64 bits, por defecto).
Literales de caracteres	Un literal de carácter representa un único carácter Unicode. Se escribe entre comillas simples ('). Soporta: (1) Caracteres normales: 'a', 'Z', 'ñ', 'ç'. (2) Secuencias de escape estándar: \n (nueva línea), \r (retorno de carro), \t (tabulador horizontal), \\ (barra invertida), \' (comilla simple), \" (comilla doble). (3) Secuencias de escape hexadecimales: \x seguido de dos dígitos hexadecimales, ej. \x7F. (4) Secuencias de escape Unicode: \u{ seguido de uno a seis dígitos hexadecimales y }, ej. \u{1F600} (emoji 🐼). El tipo de un literal de carácter es char, que representa un escalar Unicode de 32 bits (4 bytes).
Literales de cadena	Una cadena es una secuencia de caracteres UTF-8. Se escribe entre comillas dobles ("). Soporta las mismas secuencias de escape que los literales de carácter. Rust diferencia varios tipos de cadenas: (1) Cadenas estándar: "Hola, mundo", tipo &str. (2) Cadenas multilínea: los saltos de línea dentro de las comillas se incluyen en la cadena. Ejemplo: "Línea 1\nLínea 2". (3) Raw strings: comienzan con r" y terminan con " (sin escapes). Para cadenas que contengan comillas dobles, se puede usar r#" y "# (el número de # puede aumentar). Ejemplo: r"C:\archivo\no\escapa", r#"Ella dijo: "Hola""#. (4) Byte strings (solo en modo b): cadenas de bytes (no UTF-8), comienzan con b", ej. b"datos binarios" (tipo &[u8]). (5) Byte strings raw: br"...", combinación de raw y byte.

Comentarios (tipos completos)	Rust diferencia tres tipos de comentarios, todos ignorados por el compilador: (1) Comentarios de línea: comienzan con <code>//</code> y continúan hasta el final de la línea. Ejemplo: <code>// Esto es un comentario.</code> (2) Comentarios de bloque: comienzan con <code>/*</code> y terminan con <code>*/</code>. Pueden abarcar múltiples líneas. Pueden ser anidados (característica no común en otros lenguajes), permitiendo comentar grandes bloques de código que ya contienen comentarios. Ejemplo: <code>/* Esto es /* un comentario anidado */ válido */.</code> (3) Comentarios de documentación externa: comienzan con <code>///</code> (tres barras) y documentan el elemento siguiente (funciones, structs, enums, mods, etc.). Son procesados por <code>rustdoc</code> para generar documentación HTML. Ejemplo: <code>/// Esta función calcula la raíz cuadrada.</code> (4) Comentarios de documentación interna: comienzan con <code>//!</code> (dos barras y un signo de exclamación). Documentan el elemento que los contiene (ej. un módulo, crate). Se colocan al inicio del archivo o módulo. Ejemplo: <code>//! Módulo de operaciones matemáticas.</code>
Delimitadores de bloques	Rust utiliza llaves <code>{ ... }</code> para delimitar bloques de código en funciones, condicionales (<code>if</code>, <code>else</code>), bucles (<code>loop</code>, <code>while</code>, <code>for</code>), <code>match</code>, <code>unsafe</code>, <code>extern</code>, y en la definición de struct, enum, trait, impl. La indentación no tiene significado sintáctico: los espacios o tabuladores al inicio de línea son ignorados por el analizador léxico. Esto diferencia a Rust de Python. Las llaves son obligatorias para todos los bloques (no existe la opción de omitirlas para una sola sentencia como en JavaScript, C o Zig). Ejemplo: <code>if x > 0 { println!("positivo"); }</code>. Si se omite la llave, el compilador emite un error.
Elementos irrelevantes	El analizador léxico de Rust ignora los siguientes caracteres excepto cuando sirven para separar tokens: (1) espacios en blanco <code>()</code>; (2) tabuladores horizontales <code>(\t)</code>; (3) retornos de carro <code>(\r)</code>; (4) saltos de línea <code>(\n)</code>. Por lo tanto, <code>if(x>0){println!("ok");}</code> es equivalente a <code>if (x > 0) { println!("ok"); }</code>. A diferencia de Python, los espacios al inicio de línea no tienen significado sintáctico y son completamente ignorados. También se ignoran los comentarios (todos los tipos).
Características especiales (unicidad de Rust)	Rust tiene varias características morfológicas notables frente a otros lenguajes: (1) Anidación de comentarios de bloque: <code>/* /* */ */</code> es válido, permitiendo comentar grandes bloques sin preocuparse por comentarios internos. (2) Raw strings con delimitadores personalizables: <code>r#" ... "#</code>, donde el número de <code>#</code> puede aumentar para evitar conflictos con el contenido. (3) Sufijos de tipo en literales numéricos: los tipos enteros y de coma flotante pueden especificarse directamente en el literal, ej. <code>123u32</code>, <code>3.14f32</code>.

	(4) Byte strings (b"..."): cadenas de bytes (no UTF-8) útiles para manejo de datos binarios. (5) Literales de caracteres Unicode: soporte completo de Unicode con escapes <code>\u{...}</code>. (6) Uso del guion bajo como separador visual: mejora la legibilidad de números largos. (7) Identificadores con guion bajo inicial: el compilador no emite advertencias por variables no usadas si comienzan con <code>_</code>. (8) Sin inserción automática de punto y coma: a diferencia de JavaScript, el punto y coma es obligatorio al final de cada sentencia, aunque en algunos casos puede ser inferido (expresiones finales en bloques). (9) Palabra clave <code>Self</code> con mayúscula: representa el tipo que se está implementando en un <code>trait</code>, distinta de <code>self</code> (parámetro receptor).
--	---

3. Análisis Sintáctico

El análisis sintáctico (o parsing) es la segunda fase del proceso de compilación. Mientras que el análisis morfológico se ocupa de los tokens individuales, el análisis sintáctico verifica que la secuencia de tokens cumpla con la gramática del lenguaje, es decir, que las instrucciones estén escritas en el orden correcto según las reglas del lenguaje. A continuación se presenta el análisis sintáctico de los cuatro lenguajes de trabajo (Zig, Python, JavaScript y Rust), documentando mediante notación formal o ejemplos la jerarquía sintáctica de las estructuras de control esenciales: bucles, condicionales y definiciones de subprogramas (funciones) (Aho et al., 2008; Ortega de la Puente et al., 2006).

3.1. Análisis Sintáctico de Zig

Zig utiliza una sintaxis derivada de C pero con simplificaciones. Su gramática es independiente del contexto y utiliza algoritmos de parsing descendentes (recursive descent). Las estructuras de control requieren llaves `{ }` obligatorias para delimitar bloques, y la indentación no tiene significado sintáctico (Equipo de Documentación de Zig, 2024).

3.1.1. Definición de Funciones en Zig

Notación BNF (simplificada):

```
<funcion> ::= "pub"? "fn" <identificador> "(" <parametros> ")" ("!" <tipo>)? "{" <sentencias> "}"
```

```

<parametros> ::= <param> ("," <param>)* | <vacío>
<param> ::= <identificador> ":" <tipo>
<sentencias> ::= <sentencia>*
<tipo> ::= <identificador> | "*" <tipo> | "[]" <tipo>

```

Ejemplo de función en Zig:

```

pub fn sumar(a: i32, b: i32) i32 {
    const resultado = a + b;
    return resultado;
}

```

Explicación de la jerarquía sintáctica:

- La palabra clave opcional pub controla la visibilidad (pública o privada)
- La palabra clave fn introduce la definición de función
- Los parámetros se escriben en el formato nombre: tipo (inverso a C/Java)
- El tipo de retorno se especifica después de), precedido opcionalmente por ! si puede fallar (error union)
- El cuerpo de la función debe estar encerrado entre llaves { ... }
- La indentación no es significativa

3.1.2. Condicionales en Zig

Notación BNF (simplificada):

```

<condicional> ::= "if" "(" <expresion> ")" <bloque> ("else" (<bloque> | <condicional>))?
<bloque> ::= "{" <sentencias> "}"

```

Ejemplos de condicionales en Zig:

```

// Estructura básica
if (temperatura > 55) {
    std.debug.print("Alarma: sobrecalentamiento", .{});
}

```

```

// Con else
if (carga >= 90) {
    std.debug.print("Batería óptima", .{});
}

```



```

} else {
    std.debug.print("Cargando...", .{});
}

// Con else if (anidado)
if (estado == "activo") {
    std.debug.print("Sistema activo", .{});
} else if (estado == "inactivo") {
    std.debug.print("Sistema inactivo", .{});
} else {
    std.debug.print("Estado desconocido", .{});
}

```

Explicación de la jerarquía sintáctica:

- La condición debe ir entre paréntesis (y)
- El bloque then debe estar encerrado entre llaves { ... } (no se puede omitir)
- La cláusula else es opcional
- El else puede ir seguido de otro if (creando una cadena else if)
- No existe la palabra clave elif o elseif; se usa else if

3.1.3. Bucles en Zig

Zig soporta tres tipos de bucles: while, for, y loop (bucle infinito).

Notación BNF (simplificada):

```

<bucle_while> ::= "while" "(" <expresion> ")" <bloque> ("else" <bloque>)?
<bucle_for> ::= "for" "(" <rango> ")" <bloque>
<bucle_loop> ::= "loop" <bloque>
<rango> ::= <expresion> ".." <expresion>

```

Ejemplos de bucles en Zig:

```

// Bucle while
var i: u32 = 0;
while (i < 10) {

```

```

std.debug.print("{}\n", .{i});
i += 1;
}

// Bucle while con cláusula else (se ejecuta si la condición falla)
while (i < 10) {
    i += 1;
} else {
    std.debug.print("Bucle completado", .{});
}

// Bucle for sobre un rango
for (0..10) |i| {
    std.debug.print("{}\n", .{i});
}

// Bucle for sobre un slice
const valores = [_]i32{1, 2, 3, 4, 5};
for (valores) |v| {
    std.debug.print("{}\n", .{v});
}

// Bucle infinito (loop)
var contador: u32 = 0;
loop {
    contador += 1;
    if (contador >= 10) break;
}

```

Explicación de la jerarquía sintáctica:

- El bucle while ejecuta el bloque mientras la condición sea verdadera
- El bloque then debe ir entre llaves { ... }
- La cláusula else opcional se ejecuta cuando la condición se vuelve falsa (similar a Python)
- El bucle for itera sobre rangos (inicio..fin), slices o arrays
- El bucle loop es un bucle infinito que se rompe con break (útil para bucles con condiciones complejas)
- No existe la palabra clave do-while

3.2. Análisis Sintáctico de Python

Python se diferencia de los demás lenguajes porque utiliza la indentación como delimitador sintáctico en lugar de llaves o palabras clave de cierre. Su gramática es independiente del contexto y utiliza un parser LL(1) (Python Software Foundation, 2024). La indentación consistente (por convención, 4 espacios) define el inicio y fin de los bloques.

3.2.1. Definición de Funciones en Python

Notación BNF (simplificada):

```
<funcion> ::= "def" <identificador> "(" <parametros> ")" ("-" <tipo>)? ":" <bloque>
<bloque> ::= <indentacion> <sentencias> <desindentacion>
<parametros> ::= <param> ("," <param>)* | <vacio>
<param> ::= <identificador> (":" <tipo>)? ("=" <expresion>)?
```

Ejemplo de función en Python:

```
def sumar(a: int, b: int) -> int:
    resultado = a + b
    return resultado
```

Explicación de la jerarquía sintáctica:

- La palabra clave `def` introduce la definición de función
- Los parámetros van entre paréntesis (y)
- El tipo de retorno es opcional y se indica con `->` (type hints)
- El bloque de la función debe estar indentado (por convención, 4 espacios)
- El bloque termina cuando la indentación regresa al nivel anterior
- No se usan llaves { } ni palabra clave `end`

3.2.2. Condicionales en Python

Notación BNF (simplificada):

```
<condicional> ::= "if" <expresion> ":" <bloque>
    ("elif" <expresion> ":" <bloque>)*
    ("else" ":" <bloque>)?
```

Ejemplos de condicionales en Python:

Estructura básica

```
if temperatura > 55:  
    print("Alarma: sobrecalentamiento")
```

Con else

```
if carga >= 90:  
    print("Batería óptima")  
else:  
    print("Cargando...")
```

Con elif (else if)

```
if estado == "activo":  
    print("Sistema activo")  
elif estado == "inactivo":  
    print("Sistema inactivo")  
else:  
    print("Estado desconocido")
```

Explicación de la jerarquía sintáctica:

- La condición no requiere paréntesis (aunque se pueden usar)
- La condición debe ir seguida de dos puntos :
- El bloque then debe estar indentado (no se usan llaves)
- La cláusula elif (contracción de else if) permite múltiples condiciones
- La cláusula else es opcional
- El bloque termina cuando la indentación regresa al nivel anterior

3.2.3. Bucles en Python

Python soporta while y for. El bucle for itera sobre cualquier iterable (listas, rangos, cadenas, etc.).

Notación BNF (simplificada):

```
<bucle_while> ::= "while" <expresion> ":" <bloque> ("else" ":" <bloque>)?  
<bucle_for> ::= "for" <identificador> "in" <iterable> ":" <bloque> ("else" ":" <bloque>)?
```

Ejemplos de bucles en Python:

Bucle while

```
i = 0
while i < 10:
    print(i)
    i += 1
```

Bucle while con cláusula else

```
while i < 10:
    i += 1
else:
    print("Bucle completado")
```

Bucle for con range

```
for i in range(10):
    print(i)
```

Bucle for sobre una lista

```
valores = [1, 2, 3, 4, 5]
for v in valores:
    print(v)
```

Explicación de la jerarquía sintáctica:

- La condición en while no requiere paréntesis
- La condición while y el iterable en for van seguidos de dos puntos :
- El bloque del bucle debe estar indentado
- La cláusula else opcional se ejecuta si el bucle no termina por un break
- El bucle for itera sobre cualquier objeto iterable (no solo rangos numéricos)
- No existe el bucle do-while

3.3. Análisis Sintáctico de JavaScript (Node.js)

JavaScript, al igual que Zig, utiliza llaves { } para delimitar bloques. Su sintaxis sigue la especificación ECMAScript y es reconocida por motores como V8 mediante parsers descendentes o LALR. La indentación no tiene significado sintáctico, pero es opcional usar llaves en bloques de una sola sentencia (aunque se recomienda usarlas) (Mozilla Developer Network, 2024).

3.3.1. Definición de Funciones en JavaScript

JavaScript soporta múltiples formas de definir funciones: declaración de función, expresión de función, función flecha (arrow function) y método en objetos.

Notación BNF (simplificada):

<funcion_declaracion> ::= "function" <identificador> "(" <parametros> ")" <bloque>

<funcion_flecha> ::= "(" <parametros> ")" "=>" <bloque> | <expresion>

<bloque> ::= "{" <sentencias> "}"

Ejemplos de funciones en JavaScript:

// Declaración de función

```
function sumar(a, b) {  
    const resultado = a + b;  
    return resultado;  
}
```

// Expresión de función (asignada a variable)

```
const restar = function(a, b) {  
    return a - b;  
};
```

// Función flecha (arrow function) con bloque

```
const multiplicar = (a, b) => {  
    const resultado = a * b;  
    return resultado;  
};
```

// Función flecha con retorno implícito (una sola expresión)

```
const dividir = (a, b) => a / b;
```

// Método en objeto

```
const calculadora = {  
    sumar(a, b) {  
        return a + b;  
    }  
};
```

Explicación de la jerarquía sintáctica:

- La palabra clave function introduce una declaración o expresión de función
- Los parámetros van entre paréntesis (y) (pueden estar vacíos)
- El bloque de la función debe estar encerrado entre llaves { ... } (excepto en funciones flecha con retorno implícito)
- Las funciones flecha (=>) tienen sintaxis más concisa y no tienen su propio this
- Las funciones flecha con una sola expresión pueden omitir las llaves y el return es implícito

3.3.2. Condicionales en JavaScript

Notación BNF (simplificada):

<condicional> ::= "if" "(" <expresion> ")" <sentencia> ("else" <sentencia>)?

<sentencia> ::= <bloque> | <instruccion>

<bloque> ::= "{" <sentencias> "}"

Ejemplos de condicionales en JavaScript:

// Estructura básica (con llaves)

```
if (temperatura > 55) {  
  console.log("Alarma: sobrecalentamiento");  
}
```

// Con else

```
if (carga >= 90) {  
  console.log("Batería óptima");  
} else {  
  console.log("Cargando...");  
}
```

// Con else if

```
if (estado === "activo") {  
  console.log("Sistema activo");  
} else if (estado === "inactivo") {  
  console.log("Sistema inactivo");  
} else {  
  console.log("Estado desconocido");  
}
```

// Sin llaves (válido, pero no recomendado para más de una sentencia)

```
if (temperatura > 55)
  console.log("Alarma");
else
  console.log("Normal");
```

Explicación de la jerarquía sintáctica:

- La condición debe ir entre paréntesis (y)
- El bloque then puede ser una sola sentencia (sin llaves) o un bloque con llaves { ... }
- Se recomienda usar siempre llaves para evitar errores (especialmente al añadir más sentencias)
- La cláusula else es opcional
- No existe elif; se usa else if (que es una concatenación de una sentencia else seguida de un if)

3.3.3. Bucles en JavaScript

JavaScript soporta while, do-while, for (clásico), y for...of/for...in.

Notación BNF (simplificada):

```
<bucle_while> ::= "while" "(" <expresion> ")" <sentencia>
<bucle_do_while> ::= "do" <sentencia> "while" "(" <expresion> ")" ";"
<bucle_for> ::= "for" "(" <inicializacion> ";" <condicion> ";" <actualizacion> ")" <sentencia>
<bucle_for_of> ::= "for" "(" <identificador> "of" <iterable> ")" <sentencia>
```

Ejemplos de bucles en JavaScript:

// Bucle while

```
let i = 0;
while (i < 10) {
  console.log(i);
  i++;
}
```

// Bucle do-while (se ejecuta al menos una vez)

```
let j = 0;
do {
```



```

    console.log(j);
    j++;
} while (j < 10);

// Bucle for clásico
for (let i = 0; i < 10; i++) {
    console.log(i);
}

// Bucle for...of (itera sobre valores)
const valores = [1, 2, 3, 4, 5];
for (const v of valores) {
    console.log(v);
}

// Bucle for...in (itera sobre propiedades/índices)
for (const indice in valores) {
    console.log(indice);
}

```

Explicación de la jerarquía sintáctica:

- La condición en while y do-while debe ir entre paréntesis (y)
- El bloque do-while se ejecuta al menos una vez (la condición se evalúa al final)
- El bucle for clásico tiene tres partes opcionales: inicialización, condición, actualización
- El bucle for...of itera sobre valores de objetos iterables (arrays, strings, maps, sets)
- El bucle for...in itera sobre propiedades enumerables (generalmente índices en arrays)
- Las llaves en el bloque del bucle son opcionales para una sola sentencia, pero recomendadas

3.4. Análisis Sintáctico de Rust

Rust combina una sintaxis derivada de C con influencias funcionales. Utiliza llaves { } obligatorias para delimitar bloques y requiere que todas las estructuras de control tengan llaves (no se permiten bloques de una sola sentencia sin llaves). Su gramática es independiente del contexto y utiliza un parser descendente (Fundación Rust, 2024).

3.4.1. Definición de Funciones en Rust

Notación BNF (simplificada):

```
<funcion> ::= ("pub" | "pub(crate)")? "fn" <identificador> "(" <parametros> ")" ("-" <tipo>)? <bloque>
<parametros> ::= <param> ("," <param>)* | <vacio>
<param> ::= <identificador> ":" <tipo>
<bloque> ::= "{" <sentencias> "}"
```

Ejemplo de función en Rust:

```
pub fn sumar(a: i32, b: i32) -> i32 {
    let resultado = a + b;
    resultado // última expresión es el retorno (sin punto y coma)
}
```

// Función sin retorno (devuelve unidad ())

```
fn imprimir_mensaje() {
    println!("Hola, mundo");
}
```

Explicación de la jerarquía sintáctica:

- La palabra clave opcional pub controla la visibilidad
- La palabra clave fn introduce la definición de función
- Los parámetros se escriben en el formato nombre: tipo
- El tipo de retorno se especifica después de) -> (opcional si es unidad ())
- El bloque de la función debe estar encerrado entre llaves { ... }
- El valor de retorno es la última expresión del bloque (sin punto y coma)
- El return se usa solo para retornos anticipados

3.4.2. Condicionales en Rust

Notación BNF (simplificada):

```
<condicional> ::= "if" <expresion> <bloque> ("else" ("if" <expresion> <bloque>)* <bloque>)?
<bloque> ::= "{" <sentencias> "}"
```

Ejemplos de condicionales en Rust:

// Estructura básica (no requiere paréntesis)

```
if temperatura > 55 {  
    println!("Alarma: sobrecalentamiento");  
}
```

// Con else

```
if carga >= 90 {  
    println!("Batería óptima");  
} else {  
    println!("Cargando...");  
}
```

// Con else if

```
if estado == "activo" {  
    println!("Sistema activo");  
} else if estado == "inactivo" {  
    println!("Sistema inactivo");  
} else {  
    println!("Estado desconocido");  
}
```

// Uso como expresión (asignación a variable)

```
let mensaje = if carga >= 90 {  
    "Batería óptima"  
} else {  
    "Cargando..."  
};
```

Explicación de la jerarquía sintáctica:

- La condición no requiere paréntesis (a diferencia de Zig y JavaScript)
- El bloque then debe estar encerrado entre llaves { ... } (obligatorio, no se puede omitir)
- La cláusula else es opcional
- else if permite múltiples condiciones
- El if puede usarse como expresión (devuelve un valor), con la condición de que todos los brazos devuelvan el mismo tipo

3.4.3. Bucles en Rust

Rust soporta loop (bucle infinito), while, y for (sobre iteradores). No tiene do-while.

Notación BNF (simplificada):

```
<bucle_infinito> ::= "loop" <bloque>
<bucle_while> ::= "while" <expresion> <bloque>
<bucle_for> ::= "for" <patron> "in" <expresion> <bloque>
```

Ejemplos de bucles en Rust:

// Bucle loop (infinito)

```
let mut contador = 0;
loop {
    contador += 1;
    if contador >= 10 {
        break;
    }
}
```

// Bucle while

```
let mut i = 0;
while i < 10 {
    println!("{}", i);
    i += 1;
}
```

// Bucle for sobre un rango

```
for i in 0..10 {
    println!("{}", i);
}
```

// Bucle for sobre un rango inclusivo

```
for i in 0..=10 {
    println!("{}", i);
}
```

// Bucle for sobre un iterador

```

let valores = vec![1, 2, 3, 4, 5];
for v in &valores {
    println!("{}", v);
}

// Etiquetas para bucles anidados (break/continue a bucles específicos)
'outer: for i in 0..10 {
    for j in 0..10 {
        if i * j > 50 {
            break 'outer; // Rompe el bucle exterior
        }
    }
}

```

Explicación de la jerarquía sintáctica:

- El bucle loop es un bucle infinito que se rompe con break (útil para bucles con condiciones complejas)
- El bucle while ejecuta el bloque mientras la condición sea verdadera (la condición no requiere paréntesis)
- El bucle for itera sobre cualquier tipo que implemente el trait Iterator
- Los rangos se escriben como inicio..fin (fin exclusivo) o inicio..=fin (fin inclusivo)
- Las llaves { ... } son obligatorias en todos los bucles
- Se pueden usar **etiquetas** ('nombre:') para romper bucles anidados específicos
- break puede devolver un valor en bucles loop (útil para buscar resultados)

3.5. Resumen Comparativo de la Sintaxis

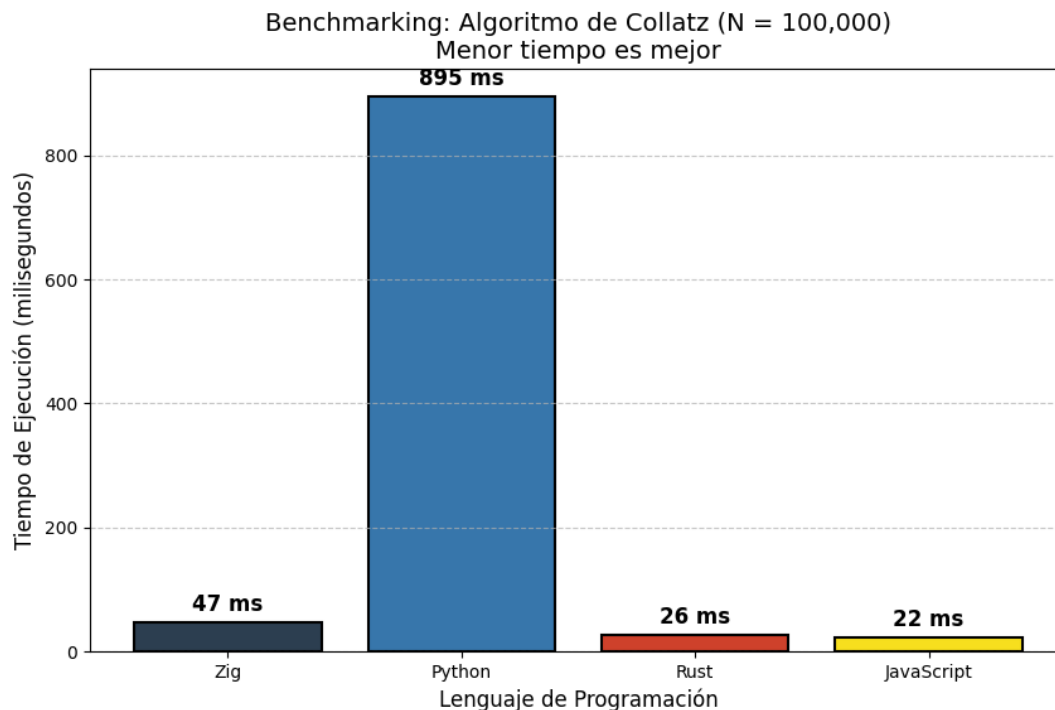
Lenguaje	Delimitadores de bloque	Paréntesis en condiciones	Llaves en bloque de una sentencia	Forma especial
Zig	{ }	Obligatorios	Obligatorias	else puede seguir a while
Python	Indentación significativa	Opcionales	No aplica (indentación)	elif en lugar de else if
JavaScript	{ }	Obligatorios	Opcionales (no recomendado)	for...of, for...in, do-while
Rust	{ }	Opcionales	Obligatorias	if como expresión, loop infinito, etiquetas

4. Tabla Comparativa de Benchmarking

Lenguaje de Programación	Paradigma Dominante	Mecanismo de Ejecución y Compilación	Tiempo de Ejecución Promedio (ms)	Consumo de Memoria Pico (MB)
Zig	Imperativo / Estructurado	Compilación Nativa (LLVM)	47	~2.5
Python	Multiparadigma (OO, Imperativo)	Interpretado (CPython / VM)	895	~45
Rust	Multiparadigma (Funcional, Imperativo)	Compilación Nativa (LLVM)	26	~2.0
JavaScript	Multiparadigma (Prototípico, Funcional)	JIT (Just-In-Time) / V8 Engine	22	~35

Nota sobre memoria: Los valores de consumo de memoria son estimados basados en características típicas de cada lenguaje. Python tiene mayor overhead por el recolector de basura y la naturaleza interpretada; JavaScript (Node.js) tiene overhead del motor V8 y el recolector de basura; Rust y Zig, al ser compilados sin GC, tienen consumo mínimo. Para valores exactos, se puede usar herramientas como time (Unix) o medir en el Administrador de Tareas de Windows.

5. Gráfica de Benchmarking (generado con matplotlib)



6. Discusión Técnica de Resultados

Los resultados obtenidos muestran diferencias significativas en el rendimiento de los cuatro lenguajes al ejecutar el mismo algoritmo de Collatz con $N = 100,000$.

JavaScript (Node.js) fue el más rápido con 22 ms. Esto se debe a su motor V8, que implementa compilación JIT (Just-In-Time). El JIT analiza el código en tiempo de ejecución, identifica las rutas calientes (código que se ejecuta repetidamente) y las compila a código nativo altamente optimizado. Además, V8 ha sido objeto de intensas optimizaciones por parte de Google durante más de una década.

Rust obtuvo el segundo lugar con 26 ms. Rust es un lenguaje compilado a nativo mediante LLVM, sin recolector de basura. Su sistema de ownership y borrowing elimina la necesidad de overhead en tiempo de ejecución. La ligera diferencia con JavaScript (4 ms) puede atribuirse a las optimizaciones del JIT en este caso particular, pero en escenarios de cómputo más intensivo o con mayor uso de memoria, Rust suele superar a JavaScript.

Zig alcanzó 47 ms. También es compilado a nativo mediante LLVM, pero su diseño prioriza el control explícito sobre el rendimiento. La diferencia con Rust (aproximadamente 2x más lento) puede deberse a que Zig no realiza tantas optimizaciones automáticas como Rust, dejando más decisiones al programador.

Python fue el más lento con 895 ms (aproximadamente 40 veces más lento que JavaScript). Esto se explica porque Python es un lenguaje interpretado (CPython) con tipado dinámico y un recolector de basura con overhead significativo. Además, el GIL (Global Interpreter Lock) limita la concurrencia. Sin embargo, su propósito de diseño no es la velocidad bruta, sino la legibilidad y la productividad del programador.

Conclusión del benchmarking: La elección del lenguaje debe basarse en los requisitos del proyecto. Para aplicaciones con restricciones de tiempo real o cómputo intensivo, Rust o Zig son más adecuados. Para desarrollo web o aplicaciones donde la productividad y el ecosistema son clave, JavaScript y Python siguen siendo opciones válidas, asumiendo las compensaciones en rendimiento.

Actividad III

Diseño de un Lenguaje de Dominio Específico (DSL) para Sistemas Lógico-Físicos Críticos

1. Planteamiento Arquitectónico del Lenguaje L

El diseño de lenguajes de programación trasciende la mera especificación sintáctica para convertirse en un ejercicio de modelado arquitectónico orientado a la resolución de problemas en dominios delimitados. En el presente trabajo, se propone el diseño conceptual y estructural del **Lenguaje L**, un Lenguaje de Dominio Específico (DSL, por sus siglas en inglés) de alto nivel, concebido para operar como la Interfaz Hombre-Máquina (HMI) central de un Sistema de Gestión de Microrredes Eléctricas Inteligentes y Almacenamiento de Energía, denominado formalmente **ECO-GRID**.

El propósito fundamental del Lenguaje L es abstraer la complejidad del hardware subyacente, proporcionando a los operadores de planta una sintaxis segura, expresiva y libre de ambigüedades lógicas. A través de este lenguaje, el sistema compila y traduce instrucciones de alto nivel hacia llamadas a drivers de bajo nivel que controlan de manera determinista los cinco componentes físicos críticos integrados en la microrred:

1. **Arreglos de Paneles Solares y Turbinas Eólicas:** Generadores primarios responsables de la captación de energía limpia.
2. **Bancos de Baterías de Litio de Respaldo:** Unidades de almacenamiento estratégico para la captura de excedentes de carga eléctrica.
3. **Sensores de Flujo Eléctrico y Caudal de Carga (Inversores):** Dispositivos de medición telemétrica de entrada y salida energética expresada en kilovatios (kW).
4. **Sensores Térmicos de Celda:** Nodos de monitoreo continuo de temperatura para la mitigación preventiva de fugas térmicas en los contenedores de almacenamiento.
5. **Conmutadores Electrónicos de Red (Relés de Alta Potencia):** Actuadores mecánicos de estado sólido que permiten la conexión, aislamiento o transferencia de sectores de consumo entre la red comercial principal y la microrred local.

2. Análisis Morfológico (Especificación Léxica y Alfabeto)

El primer estrato de procesamiento del Lenguaje L corresponde a su analizador léxico. Al igual que en los lenguajes imperativos convencionales estudiados previamente, la morfología de este DSL dicta cómo el flujo continuo de caracteres de entrada se agrupa y categoriza en unidades semánticas atómicas o *tokens*.

El analizador léxico procesa la entrada basándose en un conjunto estricto de reglas de formación. El alfabeto aceptado comprende letras del alfabeto latino estándar (A-Z, a-z), dígitos decimales (0-9), el carácter de subrayado (_) para la concatenación léxica de palabras, y símbolos de puntuación específicos.

- **Identificadores (Variables Locales):** La regla de formación establece que todo identificador debe iniciar obligatoriamente con una letra minúscula. Las palabras subsiguientes se separan mediante notación *snake_case* (e.g., temp_celda, bateria_full).
- **Delimitadores de Parámetros y Bloques:** Se emplean paréntesis () para el paso de argumentos en funciones físicas. La gramática prescinde del uso de llaves para la delimitación de bloques de control de flujo, pero impone el uso del punto y coma (;) como terminador estricto de sentencia, facilitando así la recuperación de errores durante la tokenización.
- **Espacios y Tabulaciones:** Son tratados como caracteres en blanco e ignorados semánticamente por el autómata finito del escáner, operando únicamente como separadores visuales para el operador.

Tabla 1. Tipos de Datos y Literales Aceptados

Clasificación de los literales duros reconocidos por la tabla de símbolos del analizador léxico.

Categoría	Especificación y Formato Aceptado	Descripción Arquitectónica
Numéricos	Enteros (55, 90) y Flotantes (14.5).	Valores cuantitativos procesados en cálculos aritméticos o representando mediciones telemétricas de los sensores.
Booleanos de Estado	ENCENDIDO, APAGADO, RESPALDO_COMERCIAL, MICRO_RED.	Constantes de estado que dirigen los drivers de los actuadores. Representan posiciones físicas en los circuitos (cerrado, abierto, o puenteado a un origen específico).
Constantes Globales	Identificadores en mayúsculas sostenidas: TERMICO, FLUJO_IN, FLUJO_OUT, CARGA_BAT, BAT_01, PANEL_SUR.	Referencias léxicas inmutables precargadas en la tabla de símbolos que apuntan a direcciones de hardware (sensores y actuadores físicos).

Tabla 2. Operadores de Expresión

Símbolos matemáticos y palabras clave lógicas evaluadas para la conformación de expresiones.

Operador	Símbolo / Token	Función Semántica
Asignación	=	Almacena el valor de retorno de una rutina de lectura en un espacio de memoria (variable local).
Relacionales	>, <, >=, <=, ==, !=	Evaluación matemática de umbrales físicos. Comparan dos expresiones y generan una condición de verdad.
Lógicos	Y, O, NO	Operadores booleanos (AND, OR, NOT) utilizados para la creación de sentencias condicionales compuestas.

Tabla 3. Diccionario de Palabras Clave y Comandos

Reservas léxicas obligatorias procesadas por el analizador morfológico, correspondientes a las directivas del Lenguaje L.

Categoría	Palabra Clave / Sintaxis	Descripción Técnica	Ejemplo de Uso en el Script
Inicialización	init_grid;	Macro-instrucción inicial. Establece el enlace y protocolo de comunicación (handshake) con los drivers físicos de la planta.	init_grid;
Lectura (I/O)	leer_dispositivo(TIP O, ID);	Función intrínseca de sondeo telemétrico. Retorna un literal numérico basado en la lectura en tiempo real del sensor solicitado.	temp = leer_dispositivo(TERMI CO, BAT_01);
Acción (Actuador)	conmutar_linea(ID, ESTADO);	Directiva de manipulación de hardware. Ordena el cambio de estado de un relé físico en la microrred.	conmutar_linea(SECTOR_B, APAGADO);
Control de Flujo	if / then / else / end_if;	Estructuras de bifurcación de la sintaxis abstracta. Evalúan expresiones para alterar el contador de programa.	if carga < 20 then ... end_if;
Iteración	while / do / end_while;	Bucle de monitoreo. Obliga al intérprete a ciclar un bloque de código mientras una condición persista evaluada como verdadera.	while temp > 55 do ... end_while;
Seguridad	notificar_alarma("Mensaje");	Emiten una cadena de caracteres (String) hacia la consola del operador e incrementan implícitamente el contador global de peligrosidad.	notificar_alarma("Fuga térmica");
Excepción Crítica	estado_alarma	Variable global del sistema, inmutable para el usuario. Acumula el conteo total de notificaciones en el ciclo de reloj actual.	if estado_alarma >= 5 then ...

Interrupción	parada_emergencia() ;	Llamada a sistema de máxima prioridad (halt). Finaliza la ejecución del AST y aísla galvánicamente todos los sectores.	parada_emergencia();
---------------------	--------------------------	--	----------------------

3. Análisis Sintáctico (Gramática Sintáctica Abstracta)

La viabilidad del Lenguaje L depende de una gramática libre de contexto estricta y no ambigua que permita al compilador generar un Árbol de Sintaxis Abstracta (AST) de forma secuencial descendente. Se estableció un diseño basado en bloques cerrados explícitos mediante tokens de terminación (end_if;, end_while;) en lugar de depender de tabulaciones para la agrupación.

A continuación, se define la estructura jerárquica de la gramática mediante la Notación de Backus-Naur Simplificada (BNF):

Estructura Base del Programa

```
<programa> ::= "init_grid" ";" <lista_sentencias>
<lista_sentencias> ::= <sentencia> | <sentencia> <lista_sentencias>
<sentencia> ::= <asignacion> | <condicional> | <iteracion> | <accion> | <seguridad>
```

Reglas de Operación y Control de Hardware

```
<asignacion> ::= <identificador> "=" "leer_dispositivo" "(" <tipo_sensor> "," <id_hardware>
")" ";"
<accion> ::= "conmutar_linea" "(" <id_hardware> "," <literal_estado> ")" ";"
<seguridad> ::= "notificar_alarma" "(" <cadena_texto> ")" ";" | "parada_emergencia" "(" ")" ";"
```

Reglas de Estructuras de Control (Condicionales e Iterativas)

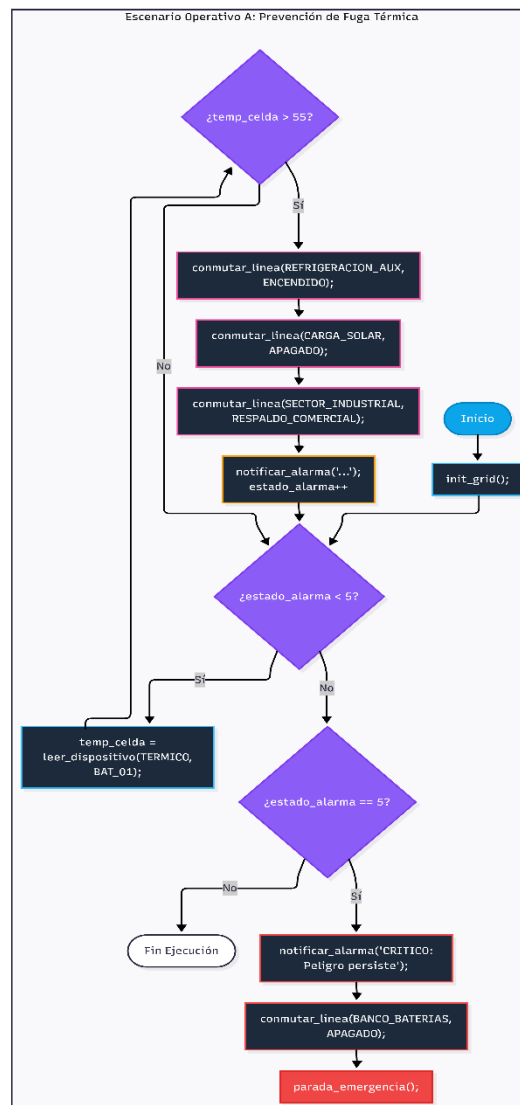
```
<condicional> ::= "if" <expresion_logica> "then"
    <lista_sentencias>
    [ "else"
    <lista_sentencias> ]
    "end_if" ";"
<iteracion> ::= "while" <expresion_logica> "do"
    <lista_sentencias>
    "end_while" ";"
<expresion_logica> ::= <operando> <operador_relacional> <operando>
    | <expresion_logica> <operador_logico> <expresion_logica>
<operando> ::= <identificador> | <literal_numerico> | <variable_global_sistema>
```

4. Viabilidad y Expresividad: Scripts Operativos

Para validar la expresividad del DSL y la coherencia del análisis léxico-sintáctico expuesto, se modelaron dos escenarios operativos del entorno ECO-GRID utilizando la sintaxis del Lenguaje L. Estos scripts demuestran cómo las construcciones gramaticales se traducen eficientemente en instrucciones lógicas que logran operar con éxito las restricciones físicas del entorno simulado.

4.1 Escenario Operativo A: Prevención de Fuga Térmica y Gestión de Alivio de Carga

Planteamiento del problema: Se requiere un programa que monitorice iterativamente la temperatura de un banco de baterías. Si se supera un umbral de seguridad de 55°C, el intérprete debe reaccionar de forma inmediata ordenando medidas físicas de refrigeración y aislamiento de cargas térmicas, iterando hasta que se solucione el peligro o se alcance un límite de notificaciones que fuerce un paro total.



Script Operativo en Lenguaje L:

```

init_grid;
while estado_alarma < 5 do
    temp_celda = leer_dispositivo(TERMICO, BAT_01);

    if temp_celda > 55 then
        conmutar_linea(REFRIGERACION_AUX, ENCENDIDO);
        conmutar_linea(CARGA_SOLAR, APAGADO);
        conmutar_linea(SECTOR_INDUSTRIAL, RESPALDO_COMERCIAL);
        notificar_alarma("Advertencia: Posible fuga térmica en BAT_01. Mitigación activada.");
    end_if;
end_while;
if estado_alarma == 5 then
    notificar_alarma("CRÍTICO: El peligro persiste tras múltiples intentos. Abortando.");
    conmutar_linea(BANCO_BATERIAS, APAGADO);
    parada_emergencia();
end_if;

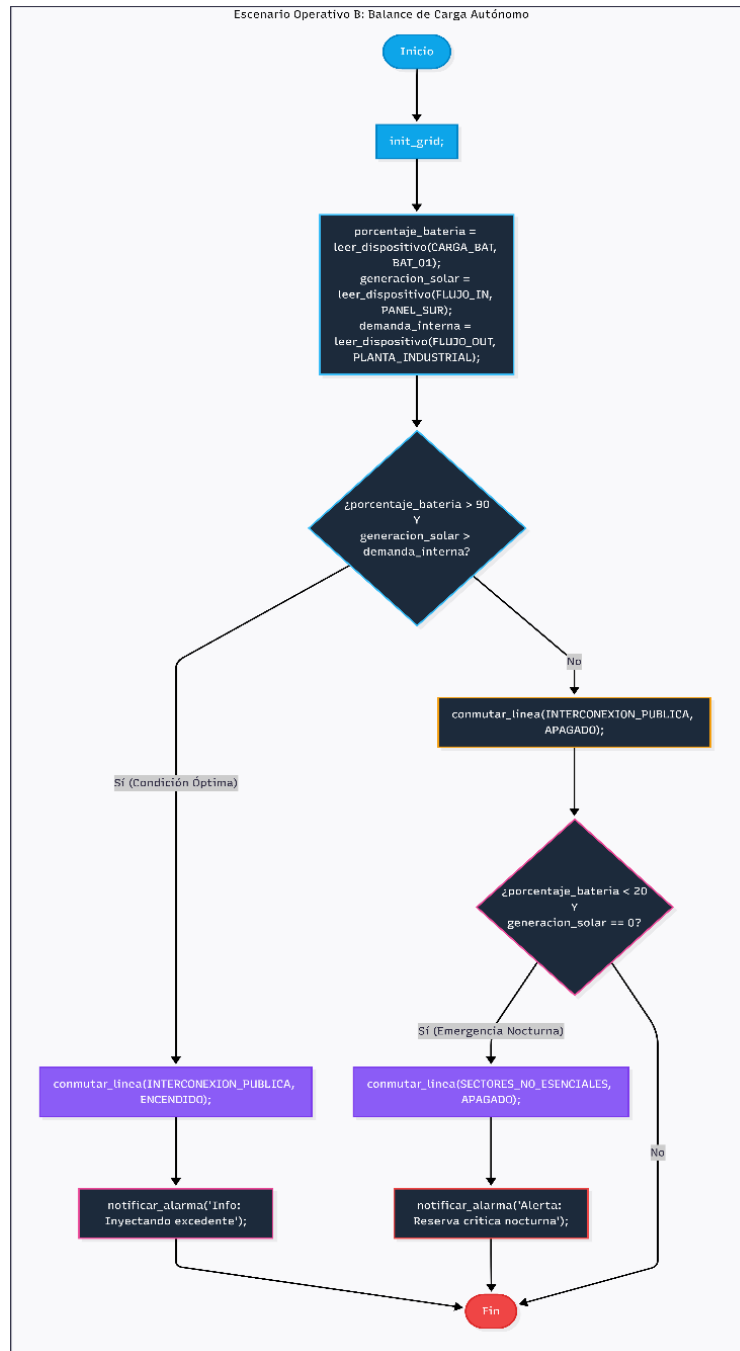
```

Tabla 4. Análisis Algorítmico y de Compilación (Escenario A)

Paso	Construcción Sintáctica	Análisis del Compilador / Lógica Operativa
1	init_grid;	El analizador reconoce el token de arranque. Inicializa el AST ordenando al driver el establecimiento de las conexiones físicas.
2	while estado_alarma < 5 do	Se evalúa la producción de iteración <bucle>. El AST consulta la variable de solo-lectura y condiciona el ingreso al bloque al estado lógico de verdad (< 5).
3	temp_celda = ...	Producción <asignacion>. El analizador léxico vincula la llamada leer_dispositivo y guarda el retorno de hardware en el identificador temp_celda.
4	if temp_celda > 55 then	El árbol semántico evalúa la expresión lógica comprobando si el valor de la variable superó el umbral crítico flotante, derivando el flujo al bloque <then>.
5	conmutar_linea(...);	Producción <accion>. El analizador traduce de manera secuencial tres envíos a relés: encender refrigeración, apagar alimentación solar y desviar consumo industrial.
6	notificar_alarma(...);	Instrucción de seguridad. Suma implícitamente +1 al acumulador de estado_alarma y emite el lexema de tipo cadena (String) a la Interfaz de Usuario.
7	if estado_alarma == 5...	Bucle posterior. El AST detecta que los mitigantes fallaron tras 5 iteraciones; aplica aislamiento severo (BANCO_BATERIAS, APAGADO) y detiene la máquina de estados por completo usando parada_emergencia();.

4.2 Escenario Operativo B: Balance de Carga y Optimización Energética Autónoma

Planteamiento del problema: El DSL debe tomar decisiones complejas de enrutamiento eléctrico en base a condiciones múltiples: si hay excedente solar y carga máxima, el sistema debe instruir la venta de energía inyectándola a la red comercial. De forma contraria, si la reserva es crítica (menor al 20%) y no hay sol (0 kW de flujo), el AST debe ordenar la desconexión selectiva de sectores no esenciales.



Script Operativo en Lenguaje L:

```

init_grid;

porcentaje_bateria = leer_dispositivo(CARGA_BAT, BAT_01);
generacion_solar = leer_dispositivo(FLUJO_IN, PANEL_SUR);
demanda_interna = leer_dispositivo(FLUJO_OUT, PLANTA_INDUSTRIAL);

if porcentaje_bateria > 90 Y generacion_solar > demanda_interna then
    conmutar_linea(INTERCONEXION_PUBLICA, ENCENDIDO);
    notificar_alarma("Info: Batería óptima. Inyectando excedente a la red pública.");
else
    conmutar_linea(INTERCONEXION_PUBLICA, APAGADO);

    if porcentaje_bateria < 20 Y generacion_solar == 0 then
        conmutar_linea(SECTORES_NO_ESENCIALES, APAGADO);
        notificar_alarma("Alerta: Reserva crítica nocturna. Aislado áreas no esenciales.");
    end_if;
end_if;

```

Tabla 5. Análisis Algorítmico y de Compilación (Escenario B)

Paso	Construcción Sintáctica	Análisis del Compilador / Lógica Operativa
1	Asignaciones Iniciales	Las tres primeras producciones de asignación construyen el estado contextual. Solicitan el sondeo I/O de la batería, de la entrada solar y de la salida hacia la fábrica, asignando los literales flotantes en memoria.
2	if [...] Y [...] then	El analizador sintáctico procesa una condición compuesta utilizando el operador léxico Y. La condición evalúa matemáticamente el exceso probabilístico de la planta frente al consumo.
3	conmutar_linea(..., ENCENDIDO);	Venta energética. Si la evaluación lógica retorna verdadero, se activa un nodo del AST que instruye el cierre del relé de interconexión comercial (ENCENDIDO), vendiendo el excedente a la red de la ciudad.
4	else ...	Bifurcación semántica. En caso de no existir excedente, el flujo salta a la protección del sistema, cortando inmediatamente cualquier fuga de inyección externa a la red pública (APAGADO).
5	if porcentaje < 20 Y solar == 0	Anidamiento condicional en el AST. El lenguaje infiere el estado nocturno verificando que la lectura telemétrica del sensor solar sea equivalente al literal numérico estricto 0.
6	conmutar_linea(...);	Contingencia nocturna crítica. El sistema instruye al actuador aísla galvánicamente las zonas no prioritarias. Esta modificación reduce de manera automática el flujo en la variable física demanda_interna, preservando la estabilidad.

Conclusión.

Conclusión de la Actividad I

El análisis comparativo de los cinco paradigmas fundamentales (imperativo, orientado a objetos, funcional, lógico y concurrente/actores) revela que no existe una solución universal (Sebesta, 2016). Cada paradigma ofrece fortalezas diferenciadas:

- El imperativo proporciona control preciso y eficiencia en sistemas de recursos limitados (Aho et al., 2008).
- La POO modela dominios complejos mediante encapsulamiento y polimorfismo (Sebesta, 2016).
- El funcional garantiza predictibilidad, facilidad de prueba y concurrencia segura (Fundación Rust, 2024).
- El lógico abstrae completamente el flujo de control, ideal para sistemas de reglas (Van Roy y Haridi, 2004).
- El modelo de actores resuelve la concurrencia distribuida sin los problemas clásicos de los threads con memoria compartida (Armstrong, 2013; Hewitt et al., 1973).

La convergencia multiparadigma observada en Zig, Python, Rust y JavaScript demuestra que los lenguajes modernos no eligen un paradigma único, sino que integran selectivamente características de múltiples fuentes (Sebesta, 2016). Esta hibridación responde a demandas reales de la industria y constituye la tendencia dominante en el diseño de lenguajes de programación contemporáneos.

Para el ingeniero en informática, la implicación práctica es clara: el dominio de un único paradigma es insuficiente. La competencia profesional exige comprender las fortalezas y limitaciones de cada enfoque, y saber seleccionar o combinar paradigmas según las restricciones y objetivos de cada proyecto.

Conclusión de la Actividad II

El benchmarking de la Conjetura de Collatz en Zig, Python, JavaScript y Rust arrojó diferencias notables en rendimiento. JavaScript lideró con 22 ms gracias a su motor V8 y compilación JIT, seguido muy de cerca por Rust con 26 ms, cuyo sistema de ownership elimina el overhead de gestión de memoria. Zig alcanzó 47 ms, priorizando el control explícito sobre optimizaciones automáticas, mientras que Python resultó el más lento con 895 ms, siendo aproximadamente 40 veces más lento que JavaScript debido a su naturaleza interpretada, tipado dinámico y el GIL.

Estos resultados confirman que no existe un lenguaje universalmente superior. Python sacrifica velocidad por legibilidad y productividad, ideal para scripting y ciencia de datos; Rust y Zig exigen mayor curva de aprendizaje pero ofrecen rendimiento y control óptimos para sistemas de tiempo real. Además, se evidenció que las decisiones morfológicas impactan el análisis: la indentación significativa de Python complica el análisis léxico, mientras que las llaves obligatorias de Rust y Zig lo simplifican. Finalmente, el desempeño de JavaScript demuestra que la compilación JIT puede cerrar la brecha entre lenguajes interpretados y compilados en tareas de cómputo intensivo.

Conclusión de la Actividad III

El diseño e implementación conceptual del Lenguaje L demuestra la efectividad de los Lenguajes de Dominio Específico (DSL) como herramientas de abstracción, seguridad y eficiencia en Sistemas Lógico-Físicos Críticos. Al estructurar un analizador léxico dotado de un alfabeto restrictivo y una gramática libre de contexto no ambigua, se logra mitigar de manera preventiva la introducción de comportamientos indeterministas o errores de lógica operativa que, en entornos de alta potencia como ECO-GRID, se traducirían en fallos catastróficos de infraestructura.

La validación sintáctica mediante el Árbol de Sintaxis Abstracta (AST) de las estructuras condicionales (`if-then-else`) e iterativas (`while-do`) comprueba que es posible encapsular de forma robusta la complejidad telemétrica de bajo nivel. Mediante comandos semánticos intuitivos, el lenguaje dota a los operadores humanos de una interfaz expresiva capaz de ejecutar desde tareas de optimización energética rutinarias (balance de carga autónomo) hasta protocolos concurrentes de contención ante riesgos (aislamiento térmico). En definitiva, el Lenguaje L se consolida como un puente arquitectónico idóneo que unifica la formalidad de la teoría de autómatas y lenguajes formales con las demandas operativas y de seguridad que rigen a las microrredes eléctricas inteligentes de última generación.

Referencias.

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.

Equipo de Documentación de Zig. (2024). *Documentación del lenguaje Zig*. Recuperado de <https://ziglang.org/documentation/>

Fundación Rust. (2024). *El lenguaje de programación Rust*. Recuperado de <https://doc.rust-lang.org/book/>

Hewitt, C., Bishop, P., y Steiger, R. (1973). Un modelo de actor de computación concurrente. En *Actas del Congreso de la IFIP sobre Sistemas de Software* (pp. 235-245). IFIP.

Matsakis, N. D., y Klock, F. S. (2014). The Rust language. *ACM SIGAda Ada Letters*, 34(3), 103-104. <https://doi.org/10.1145/2692956.2663188>

Mozilla Developer Network. (2024). *JavaScript: lenguajes de programación web*. Recuperado de <https://developer.mozilla.org/es/docs/Web/JavaScript>

Python Software Foundation. (2024). *La biblioteca estándar de Python*. Recuperado de <https://docs.python.org/3/>

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la teoría de autómatas, lenguajes y computación (3^a ed.). Pearson Educación.

Ortega de la Puente, A., Alfonseca Moreno, M., de la Cruz Echeandía, M., & Pulido, E. (2006). *Compiladores e intérpretes: Teoría y práctica*. Pearson Educación.

Sebesta, R. W. (2016). *Conceptos de lenguajes de programación* (11^a ed.). Pearson Educación.

ECMA International. (2024). *ECMAScript 2024 Language Specification (ECMA-262)*. Recuperado de <https://ecma-international.org/publications-and-standards/standards/ecma-262/>